GH DEVELOPER LEARNING SERIES

2026 EDITION · PDF

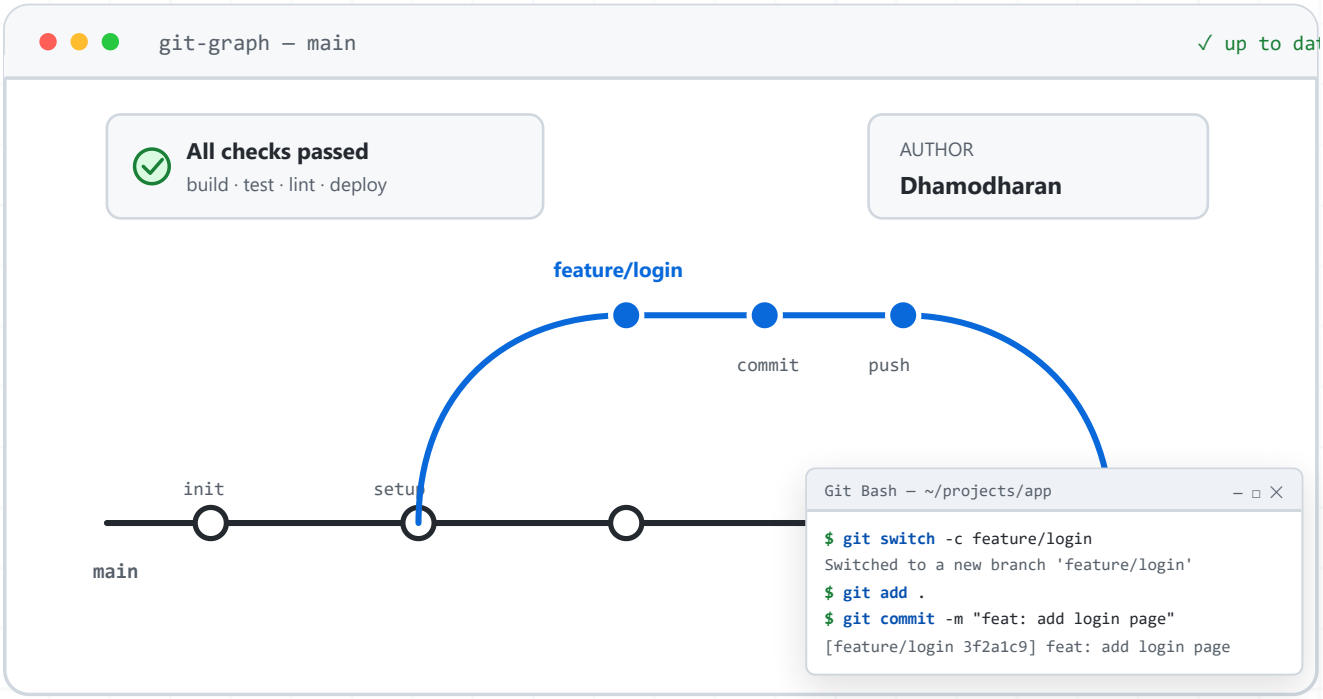
PRACTICAL · REAL-TIME · INTERVIEW-READY

GIT & GITHUB

COMPLETE DEVELOPER GUIDE

Beginner to Advanced | Practical Learning | Real-Time IT Company Workflow

13 Chapters | 150+ Topics | 100+ Commands | 22 Visual Walkthroughs | Interview Q&A



WRITTEN & ILLUSTRATED BY

Dhamodharan

Git 2.x · GitHub Actions · Pages · Security

For Students, Java Full Stack & React Developers

Clear, simple explanations · A4 Print Edition

START HERE

How to Use This Book

This book takes you from zero to **real IT company developer** level. It starts with Git basics and then covers GitHub collaboration, Actions, Security, Pages, and real team workflow, step by step. Every chapter has an explanation, a command, the expected output, an illustrative visual, and real-time usage.

- **Who is this book for?**

Students & freshers who want their first developer job. **Java Full Stack** and **React developers** who need Git for daily work. **Job seekers** preparing for Git interview questions.

- **What should you know already?**

Basic computer usage and one programming language is enough. Git experience is **not required**. All Windows commands are explained using Git Bash.

1. How every topic is explained

What & Why
The concept, and why we use it — explained simply.

Syntax & Example
Exact command, real file names, and ready-to-run example.

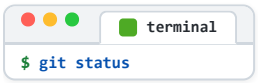
Output & Usage
Expected terminal output plus IT company real-time usage.

Important topics include **step-by-step numbered instructions**, **comparison tables**, and **common mistakes**. Each chapter ends with a quick recap and a practice task.

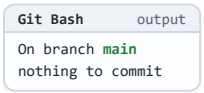
2. Visual legend

ILLUSTRATIVE MOCKUP

UI walkthrough visuals. Every GitHub, VS Code, Git Bash, and Windows screen in this book is a **clearly labelled illustrative mockup** — not a screenshot taken from the actual software. If you follow the steps, your screen will look similar. All commands, outputs, and workflows are real and runnable.



VS Code Light+ code editor. File tab, line numbers, syntax colours. You can copy a command and paste it into the terminal. No dark code blocks are used.



Terminal output. The result you should expect. If you copy the output text from your machine, it will look the same.

Best practice

Warning

Callouts. Green = follow this. Amber = avoid this mistake. Blue = concept note. Red = data loss risk.

BEFORE YOU START

Install Git, create a GitHub account, and install VS Code. Chapter 1 has the full installation steps. Create an empty folder for practice — `C:\projects\git-practice`. Every example in the book can be run from this folder.

> *"A developer who does not know Git cannot work in an IT company today. It is your daily tool — this book teaches it practically."*

CONTENTS

What's inside

13 chapters · 150+ topics · from your first `git init` to a complete company release workflow.

01 Git Fundamentals 4
Version control, Git vs GitHub, architecture, installation, configuration, first repository
Version control · Git vs SVN · 3 areas · Install · git config · git init

02 Basic Git Commands 9
The 13 commands you will type every single working day
status · add · commit · log · diff · .gitignore · restore · clean

03 GitHub Repository Management 14
Accounts, repositories, README, visibility, settings, releases
Create repo · Public vs private · README · License · Settings · Releases

04 Remote Repository 18
Connect local code to GitHub and keep both sides in sync
remote add · push · pull · fetch · clone · SSH vs HTTPS

05 Branching and Merging 23
Parallel development, merge types, conflicts, rebase, Git Flow
switch -c · fast-forward · 3-way merge · conflicts · rebase · cherry-pick

06 Advanced Git 28
Stash, reset, revert, reflog, tags, hooks, worktrees, bisect
stash · reset · revert · reflog · interactive rebase · hooks · LFS

07 Pull Requests & Collaboration 35
Fork, Pull Requests, reviews, CODEOWNERS, protected branches, Issues
Fork · Pull Request · Code review · Merge strategies · CODEOWNERS · Issues

Part 1 · Foundations
Chapters 1–4. Git mental model, daily commands, GitHub repos, and remotes.

Part 2 · Team skills
Chapters 5–7. Branching, advanced recovery, PR review, and collaboration.

Part 3 · Professional
Chapters 8–13. CI/CD, security, projects, Pages, and real company workflow.

BEFORE YOU BEGIN

INSTALL THESE FOUR
Git for Windows (includes Git Bash) · **VS Code** · **Node.js LTS** (React examples) · **JDK 17 + Maven** (Spring Boot examples). The free GitHub plan is enough.

HOW TO PRACTISE
Create a `git-practice` folder. Run the commands from every chapter inside it. If the command output does not match, search the error message in Appendix B.

CONTENTS · CONTINUED

Chapters 8 to 13 & appendices

08 **GitHub Actions & CI/CD** 41
Automate build, test, and deploy on every push and Pull Request
Workflow YAML · Triggers · Jobs · Runners · Matrix · Artifacts

09 **GitHub Security** 46
Protect your account, code, and secrets like a real company
2FA · SSH keys · PAT · Secrets · Dependabot · CodeQL

10 **GitHub Project Management** 50
Issues, labels, milestones, Projects boards, and roadmaps
Issues · Labels · Milestones · Projects · Views · Roadmap

11 **GitHub Pages & Deployment** 53
Publish a live website directly from your repository
Pages setup · Actions deploy · Custom domain · HTTPS · React SPA

12 **Professional GitHub Skills** 56
Profile README, open source, commit conventions, versioning
Profile · Open source · Semantic commits · SemVer · GitHub CLI

13 **Real-Time IT Company Workflow** 60
A complete React + Spring Boot team project, sprint by sprint
Team roles · Branch naming · PR flow · Reviews · Conflicts · Deploy

APPENDICES & PRACTICE

A **Command Cheat Sheet** 65
Every command from the book on two pages

B **Common Errors & Fixes** 67
Real error messages with fixes

C **Git & GitHub Interview Q&A** 68
12 questions asked in fresher and developer interviews

D **Practice Tasks & 7-Day Plan** 69
Ten graded tasks and a seven-day plan

E **You Are Now Git-Ready** 70
Your 30-day plan after finishing this book

STUDY PLAN

Week 1: Chapters 1–3 + daily command drills. **Week 2:** Chapters 4–6 + branch and conflict practice. **Week 3:** Chapters 7–9 + one Pull Request per day. **Week 4:** Chapters 10–13 + practice tasks from Appendix D.

CHAPTER 01 · FOUNDATIONS

Git Fundamentals

What version control is, how Git works, and how to create your first repository on your computer.

9 topics | Level: **Beginner** | Time: **45 min** | Tool: **Git Bash + VS Code**

In a company, 10 developers work on the same project. What happens when one developer overwrites another developer's code? How do you go back to a previous version to fix a bug? These problems are exactly why we use a **Version Control System (VCS)**. Git is now the most used VCS in the world — more than 90% of professional teams use it.

1.1 What is Version Control?

DEFINITION

A Version Control System is a software tool. It **tracks every change** made to files, records **who changed what and when**, and helps you **return to any previous version** when needed.

Simple example: you save your resume as `resume_v1.docx`, `resume_v2_final.docx`, and `resume_final_final.docx`. That is manual version control. The problems: it is hard to know which one is latest, you cannot compare old versions properly, and sharing with a team is messy. Git manages all of this automatically.

Track
Every file change, author, timestamp, and reason (commit message) is recorded.

Revert
If a bug appears, you can return to yesterday's version or restore a single file.

Collaborate
Multiple developers can work safely on the same repo using branches.

1.2 Why Git? (Features)

- **Distributed:** the full history lives on your laptop, so you can commit even without internet.
- **Fast:** almost every operation runs locally — in milliseconds.
- **Branching:** creating a branch is cheap and instant; merging is easy.
- **Data integrity:** every commit has a SHA-1 hash, so Git detects corruption.
- **Open source:** free, huge community, and support for every OS.
- **Industry standard:** GitHub, GitLab, and Bitbucket all use Git.

1.3 Git vs GitHub

This is the most common confusion for freshers. **Git is a tool** that runs on your computer. **GitHub is a website and service** that stores Git repositories in the cloud. You cannot use GitHub without Git, but you can use Git without GitHub.

Point	Git	GitHub
What	Version control software	Cloud hosting + collaboration platform
Where it runs	Local laptop / server	Browser + cloud servers
Created by	Linus Torvalds (2005)	GitHub Inc. (2008), now Microsoft
Internet needed?	No (for local work)	Yes
Main use	Track code versions	Store repos, Pull Requests, Actions, Issues
Alternatives	SVN, Mercurial	GitLab, Bitbucket, Azure DevOps

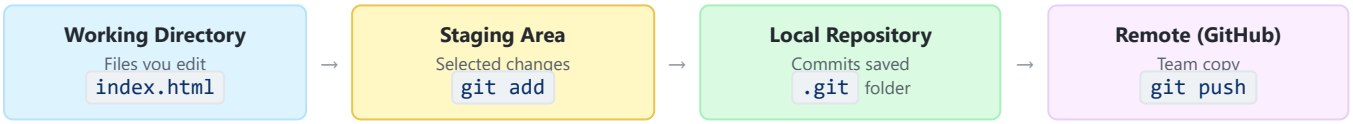
1.4 Git vs SVN

Interviewers often ask: "Why Git, and not SVN?" The answer is simple — SVN is a **centralized** system, while Git is **distributed**. If the SVN server is down, you cannot commit. With Git, the history is local, so you can keep working.

Feature	Git (Distributed)	SVN (Centralized)
Repository model	Every developer has a full repo copy	One central server repo
Commit speed	Local, very fast	Network round-trip needed
Offline work	Full support	Very limited
Branching	Lightweight, instant	Heavy directory copy
Merging	Powerful 3-way merge + rebase	Basic, conflict-prone
History	Distributed; cannot be lost easily	Central server only
Learning curve	Steeper (staging area)	Simpler concept

1.5 Git Architecture — The Three Areas

You work in three places in Git. Once this mental model is clear, 90% of the confusion disappears. You edit files in the **Working Directory**, select changes for the next commit in the **Staging Area**, and save them permanently in the **Local Repository**.



Area	Command in	What happens there
Working Directory	`edit files`	You write or modify code in VS Code. Git is not tracking it yet.
Staging Area (index)	`git add`	You select which files should go into the next commit.
Local Repository	`git commit`	The snapshot is saved permanently in the `.git` folder.
Remote Repository	`git push`	Commits are uploaded to the GitHub server and shared with the team.

MENTAL MODEL

Think of online shopping: the **Working Directory** is browsing items in a shop. The **Staging Area** is adding them to the cart. A **commit** is placing the order. A **push** is sending it to the delivery address (GitHub). You cannot place an order without adding items to the cart — just as you cannot commit new files without `git add`.

1.6 File Lifecycle — Untracked to Committed

A file in Git moves through four states. The `git status` command shows exactly which state it is in.

State	Meaning	git status says	Next action
Untracked	Git has not seen it yet; new file	`Untracked files:`	`git add`
Modified	Tracked file changed, not staged	`Changes not staged`	`git add`
Staged	Ready for the next commit	`Changes to be committed`	`git commit`
Committed	Saved in local history	`nothing to commit`	`git push`

Note: Files listed in `.gitignore` (node_modules, .env, target/) are ignored by Git — they do not even appear as untracked. You will see this in detail in Chapter 2.

1.7 Install Git on Windows

Download the installer from **git-scm.com/download/win**. You can accept the default options, but pay attention to these three choices.

1. **Download & run**
   Download `Git-2.x-64-bit.exe` and double-click it. Click **Yes** on the User Account Control prompt.

2. **Default editor**
   Select **Use Visual Studio Code as Git's default editor**. Notepad and Nano also work if you prefer.

3. **PATH option**
   Select **Git from the command line and also from 3rd-party software** (recommended) so the `git` command also works in PowerShell and CMD.

4. **Line endings**
   Select **Checkout Windows-style, commit Unix-style line endings**. This is safest for a mixed Windows and Linux team.

5. **Finish & verify**
   Click Install, open **Git Bash** from the Start menu, and type `git --version`.

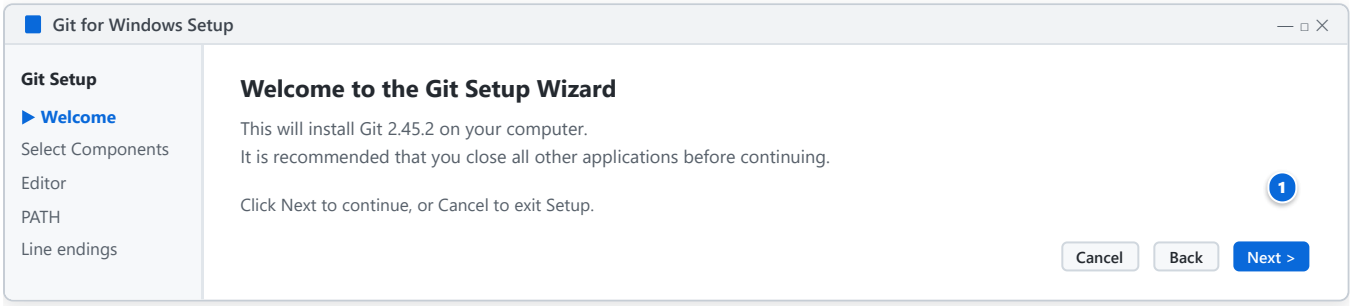


Fig 1.1 ILLUSTRATIVE MOCKUP **Git for Windows installer.** Welcome screen — click Next and install with the default options. Only the PATH option in step 3 needs a change.

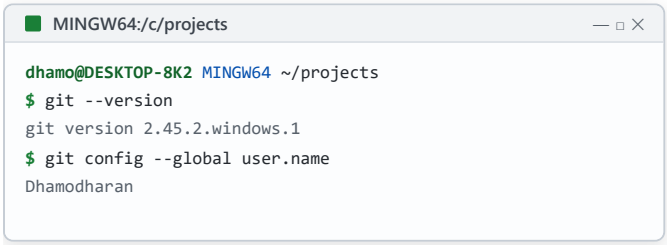


Fig 1.2 ILLUSTRATIVE MOCKUP **Git Bash after install.** If you see the version output, the install succeeded.

1.7.1 Which terminal to use?

Git Bash — recommended for beginners; Linux-style commands (`ls`, `pwd`, `cat`) work. **PowerShell / CMD** — works, but path syntax differs in some examples. **VS Code terminal** — open it with `Ctrl + ` ` and select Git Bash from the terminal dropdown. All chapters use Git Bash examples.

VERIFY CHECKLIST

`git --version` → version number
`git config --global user.name` → your name
`git help -a` → command list

1.7.2 Install on macOS & Linux (short note)

macOS: `brew install git` (Homebrew) or `xcode-select --install`. **Ubuntu/Debian:** `sudo apt update && sudo apt install git`. **Fedora:** `sudo dnf install git`. Verify with `git --version`.

1.8 Configure Git (One-Time Setup)

Do this right after installing Git. These details are saved as the author of every commit. If the email is wrong, GitHub will not show the contributions on your profile.

git-setup.sh

```
# 1. Identity — saved as the author of every commit
git config --global user.name "Dhamodharan"
git config --global user.email "dhamo@example.com"

# 2. Default branch name (GitHub default: main)
git config --global init.defaultBranch main

# 3. Line endings (Windows + Linux team)
git config --global core.autocrlf true

# 4. Default editor (VS Code)
git config --global core.editor "code --wait"

# 5. Verify everything
git config --list
```

Git Bash — verify config

```
$ git config --global user.name
Dhamodharan
$ git config --global user.email
dhamo@example.com
$ git config --list --show-origin
file:C:/Users/dhamo/.gitconfig
  user.name=Dhamodharan
  init.defaultbranch=main
```

.gitconfig — Visual Studio Code

```
[user]
    name = Dhamodharan
    email = dhamo@example.com
[init]
    defaultBranch = main
[core]
    autocrlf = true
```

Fig 1.3 ILLUSTRATIVE MOCKUP **Global config file.** Open `~/.gitconfig` in VS Code to verify it.

Common mistake 1 · Wrong email

Commits go to a private email and the green squares never appear. Use a verified email from GitHub **Settings → Emails**. Check with `git config user.email`.

Common mistake 2 · Global vs local

Your office laptop may still have a personal email set. Fix it for one repo with `git config user.email "office@company.com"` (without `--global`).

IMPORTANT NOTES

`--global` applies to all repos on your laptop. To change one repo only, remove `--global`. The config is stored in `C:\Users\<name>\.gitconfig`. Running `git config --list --show-origin` shows which file each value came from — useful for debugging proxy and SSL issues at the office.

1.8.1 Config levels — which one wins?

Level	Flag	File	Scope
System	`--system`	`C:\Program Files\Git\etc\gitconfig`	All users on the machine
Global	`--global`	`C:\Users\<name>\.gitconfig`	Your user, all repos
Local	`--local`	`.git/config`	This repo only (highest priority)

1.9 Create Your First Repository

A repository (repo) is a project folder plus Git history. You can create one in two ways: (a) `git init` in a new folder, or (b) `git clone` from GitHub (Chapter 4). Here is the first method.

1. **Create folder & initialize**
   `mkdir -p ~/projects/git-practice` → `cd ~/projects/git-practice` → `git init`. A hidden `.git` folder is created — that is the Git database. Do not delete it.
2. **Create first file**
   `echo "# Git Practice" > README.md`, or create the file in VS Code.
3. **Check status**
   `git status` — you will see README.md as untracked.
4. **Stage & commit**
   `git add README.md`, then `git commit -m "docs: add project readme"`.

git init — full first-repo flow

```
mkdir -p ~/projects/git-practice && cd ~/projects/git-practice
git init
echo "# Git Practice" > README.md
git status
git add README.md
git commit -m "docs: add project readme"
git log --oneline
```

Git Bash — expected output

```
$ git init
Initialized empty Git repository in C:/Users/dhamo/projects/git-practice/.git/
$ git status
On branch main · No commits yet
Untracked files:
        README.md
$ git add README.md
$ git commit -m "docs: add project readme"
[main (root-commit) 7f3a91c] docs: add project readme
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

Success check: if `git log --oneline` shows something like `7f3a91c docs: add project readme`, your first commit succeeded. That hash is the commit ID — you will use it later to return to a version.

REAL-TIME IT COMPANY USAGE

On your first day as a developer, the tech lead gives you a `git clone` link. But `git init` is still used daily for internal tools, scripts, and POCs. When the team creates a new microservice, the lead usually prepares the repo structure, `.gitignore`, and README template before running `git init`.

1.10 Chapter Recap & Practice

- Git = distributed VCS; GitHub = cloud platform.
- Working directory → staging → local repo → remote.
- One-time setup: name, email, default branch.

PRACTICE TASK

Create a `git-practice` folder, init Git, add 3 files (`index.html`, `style.css`, `README.md`), stage only two files, commit, and run `git status` + `git log`. Explain why the third file was not committed.

CHAPTER 02 · DAILY TOOLKIT

Basic Git Commands

This is your daily driver — 13 commands with correct syntax, expected output, and real usage.

13 commands | Level: **Beginner** | Time: **90 min** | Practice: **Required**

In Chapter 1 we ran `git init` and made the first commit. Now let us look at the complete command set. These 13 commands cover 90% of daily work — practise them in the same repo.

2.1 git init — Start Tracking

terminal

```
git init                  # new repo in the current folder
git init my-app           # create a new folder and init a repo
git init --bare repo.git  # server-side bare repo (no working files)
```

What it does: creates the hidden `.git` folder, which holds objects, refs, config, and hooks. Files are not committed just because you ran init — you still need `git add` + `git commit`.

2.2 git status — Your Best Friend

This command shows which files in the current repository are modified, staged, or untracked. It is a safe command — it changes nothing.

terminal

```
git status            # full status
git status -s         # short status (M, A, ??)
git status --branch   # branch + ahead/behind info
```

Full status — readable

```
$ git status
On branch main
Changes not staged for commit:
  modified:   index.html

Untracked files:
  style.css

no changes added to commit
```

STATUS CODES

`M` modified · `A` added · `D` deleted · `??` untracked · `UU` merge conflict. Short form: `git status -s`. Upstream info: `git status --branch`.

2.3 git add — Stage Your Changes

Before committing, you add changes to the staging area. This is the "select" step. New files are not committed without it.

terminal

```
git add index.html          # one file
git add .                   # all changes in the current folder
git add -A                  # all changes including deletes
git add -p                  # choose hunks interactively
```

COMMON MISTAKE

`git add .` also stages `node_modules/`, `.env`, and build output. Create `.gitignore` first (Section 2.10), then add. Once a password or API key is committed, removing it from GitHub history is very painful.

2.3.1 VS Code Source Control Panel

You can stage and commit without typing commands. In VS Code, click the **Source Control** icon on the left (or press `Ctrl + Shift + G`). The changes list, stage (+), discard, and commit message box are all here. This is the easiest path for beginners.

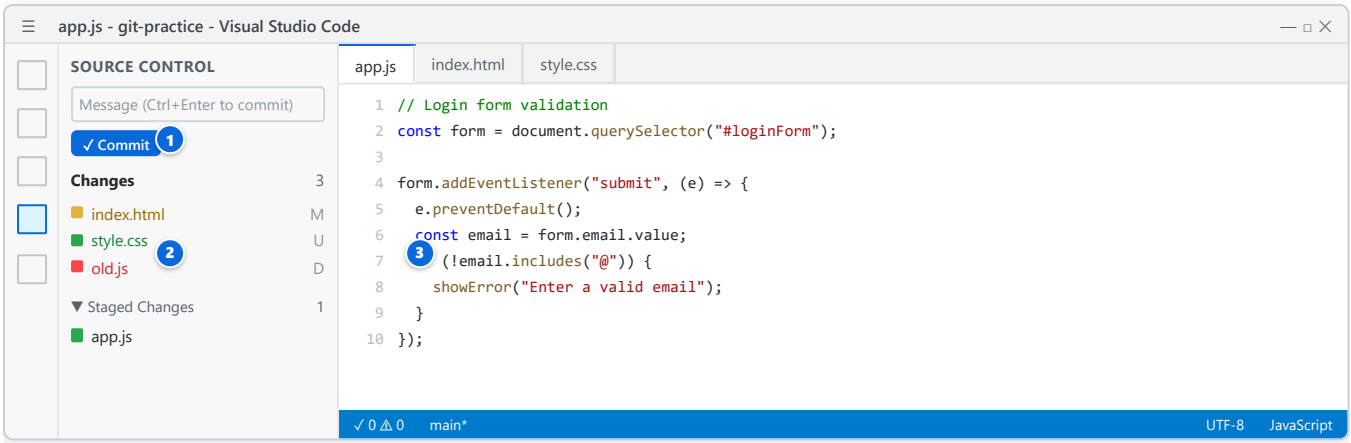


Fig 2.1 ILLUSTRATIVE MOCKUP **VS Code Source Control panel.** ① Activity bar Source Control icon. ② Commit message box + Commit button. ③ Changed files with status letters (M/U/D). Click + to stage.

2.4 git commit — Save a Snapshot

Saves staged changes as a permanent snapshot in the local repository. Every commit gets a unique ID (hash), author, date, and message. A commit is like one photo in a project album.

terminal

```
git commit -m "feat: add login form validation"
git commit -am "fix: correct email regex"   # tracked files only
git commit --amend -m "better message"      # fix last commit
git commit --allow-empty -m "chore: trigger CI"
```

Git Bash — commit output

```
$ git commit -m "feat: add login form validation"
[main 4b8e2f1] feat: add login form validation
 2 files changed, 34 insertions(+), 2 deletions(-)
 create mode 100644 js/login.js
```

Good commit message

`feat: add login form validation`
`fix: handle null user in profile API`
`docs: update setup instructions`

Bad commit message

`update` `changes` `final code` `asdf`

A reviewer cannot tell what changed, and future blame/bisect becomes useless.

Rule: one logical change = one commit. "Login page + API refactor + CSS fix" in one commit is hard to review. Type prefixes (`feat` , `fix` , `docs` , `test` , `chore`) are a team convention — detailed in Chapter 12.

Part	Example	Meaning
Type	`feat`, `fix`, `docs`	What kind of change
Scope (optional)	`feat(auth):`	Which module
Subject	`add JWT login`	Imperative, under 50 chars

2.5 git log — Read History

View the commit history. The default output is detailed; in practice, the oneline and graph options are what you use daily.

terminal

```
git log                                 # full history
git log --oneline                       # one line per commit
git log --oneline --graph --all         # branch graph
git log -5                              # last 5 commits
git log --author="Dhamodharan"
git log --since="2 weeks ago" --oneline
git log -p index.html                   # changes + history
```

Git Bash — git log --oneline --graph --all

```
* 9c1d4ab (HEAD -> main, origin/main) merge: feature/login into main
|\
| * 4b8e2f1 (feature/login) feat: add login form validation
| * 1a7c002 feat: create login page UI
|/
* 7f3a91c docs: add project readme
```

Options: `--oneline` short hash · `--stat` files changed · `-p` full diff · `--author` / `--since` filter.

2.6 git diff — Compare Changes

Verify "what did I change?" before committing. Build this habit before every code review.

terminal

```
git diff                # working dir vs staging
git diff --staged       # staging vs last commit
git diff HEAD           # working dir vs last commit
git diff main..feature  # between two branches
git diff --stat         # summary only
```

Git Bash — git diff

```
diff --git a/index.html b/index.html
@@ -12,6 +12,8 @@
   <h1>Welcome</h1>
-  <p>Old text</p>
+  <p>Updated text</p>
+  <button>Login</button>
```

HOW TO READ A DIFF

Red / minus = removed line. **Green / plus** = added line. `@@ -12,6 +12,8 @@` = 6 lines starting at line 12 in the old file, 8 lines starting at line 12 in the new file. `a/` = old version, `b/` = new version.

2.7 git show — Inspect One Commit

terminal

```
git show                    # latest commit details
git show 4b8e2f1            # specific commit
git show --stat 4b8e2f1     # files changed only
git show 4b8e2f1:index.html # file content at that commit
```

Use case: when QA says "the login page broke yesterday", run `git log -- js/login.js`, take the last change's commit ID, and run `git show <id>` to see exactly what changed. `git blame file` also shows who last changed each line.

2.8 git rm — Delete & Stage

terminal

```
git rm old.js              # delete file + stage
git rm -r assets/          # delete folder
git rm --cached .env       # stop tracking, keep local file
```

`git rm` deletes the file from the filesystem and records the deletion in the staging area. A normal `rm old.js` needs `git add -A` afterwards. **Most important:** if `.env` was committed by accident, use `git rm --cached .env` — the local file stays safe and Git simply stops tracking it.

2.9 git mv — Rename & Move

terminal

```
git mv old-name.js new-name.js
git mv src/utils.js src/helpers/utils.js
```

This renames and stages in one step. Git detects the rename internally and follows the history even though it is stored as a delete plus an add. If you rename normally, run `git add -A`.

2.10 .gitignore — Keep Noise Out

List files Git should ignore in a `.gitignore` file at the project root. This is very important — secrets, dependencies, and build output must never enter the repo.

.gitignore

```
# dependencies
node_modules/
target/
vendor/

# secrets — NEVER commit
.env
.env.local
*.pem
application-prod.properties

# build output
dist/
build/
*.class
*.log

# IDE / OS
.vscode/
.idea/
.DS_Store
Thumbs.db

# keep folder but ignore contents
uploads/*
!uploads/.gitkeep
```

Pattern	Meaning
`*.log`	All .log files, any folder
`build/`	build directory
`**/temp`	temp in any subfolder
`!keep.txt`	Exception — do not ignore
`/config.js`	Root only

ALREADY COMMITTED?
Adding `.gitignore` only ignores future changes. For a file that is already tracked: `git rm --cached file` → commit. If `node_modules` gets committed, the repo size explodes.

Pro tip: when creating a GitHub repo, select a language-based .gitignore template (Node, Java, React). `git check-ignore -v .env` shows which pattern is ignoring a file; `git status --ignored` lists ignored files. Companies usually have a shared template — copy that one.

2.11 git restore — Undo Local Changes

terminal

```
git restore index.html              # discard working changes (danger!)
git restore --staged index.html     # unstage, keep changes
git restore --source=HEAD~1 app.js  # old version content
```

DATA LOSS WARNING

`git restore index.html` permanently throws away your unsaved edits — Git cannot recover them. Double-check before discarding uncommitted work. A safe alternative is `git stash` (Chapter 6).

2.12 git clean — Remove Untracked Files

terminal

```
git clean -n          # dry run — show what would be deleted
git clean -f          # delete untracked files
git clean -fd         # untracked files + folders
git clean -fdx        # include ignored files (node_modules!)
```

Always run `-n` first. It removes only untracked files, and Git cannot restore them. With `-x`, your `.env` and `node_modules` disappear too — expect build errors afterwards.

2.13 git config Aliases — Work Faster

terminal

```
git config --global alias.st "status -s"
git config --global alias.co "checkout"
git config --global alias.br "branch"
git config --global alias.lg "log --oneline --graph --all"
# usage
git st        # same as: git status -s
git lg        # beautiful history graph
```

Situation	Correct command
Edited file, not staged yet	`git restore file` (discard) or `git stash` (keep)
Staged by mistake	`git restore --staged file`
Untracked junk files	`git clean -fd` (dry run `-n` first)
Bad commit, not pushed	`git reset --soft HEAD~1` or `--mixed`
Bad commit, already pushed	`git revert <hash>` — never reset a shared branch

CHAPTER RECAP & PRACTICE

Daily flow: `git status` → edit → `git diff` → `git add -p` → `git commit -m "..."` → `git log --oneline`. **Practice:** create 3 files in a repo, add one to `.gitignore`, compare staged and unstaged diffs, and make 2 separate commits. Then try `git restore --staged` and `git clean -n`.

CHAPTER 03 · YOUR CLOUD HOME

GitHub Repository Management

Create an account and configure your repository the right way — README, license, visibility, and releases.

11 topics | Level: **Beginner** | Time: **60 min** | Needs: **GitHub account**

Git works locally; GitHub creates a **professional home** for your code. Recruiters shortlist you based on your GitHub profile. So take the repo name, README, license, and settings seriously.

3.1 Create Your GitHub Account

1. **Sign up**
   github.com → Sign up → username, email, strong password. Pick a professional username (`dhamo-dev`), not `coolboy2005` .
2. **Verify email**
   Click the verification link in your inbox. Without it, Pull Requests, Actions, and more stay blocked.
3. **Profile setup**
   Settings → Public profile: full name, bio, location, company, website, profile picture. This is the first impression for a recruiter.
4. **Enable 2FA**
   Settings → Password and authentication → Two-factor authentication. Mandatory to contribute to a company org (Chapter 9).

3.2 Create a Repository

Top-right **+** icon → **New repository**. Fill these fields carefully.

Field	What to choose
Repository name	lowercase + hyphens: `employee-management`. Avoid spaces and underscores.
Description	One line: "React + Spring Boot employee CRUD app". This is what shows on the repo card.
Visibility	**Public** = portfolio/open source. **Private** = company/internal work.
Initialize	For a new project add README + .gitignore + license. For an existing local repo, **don't tick anything**.

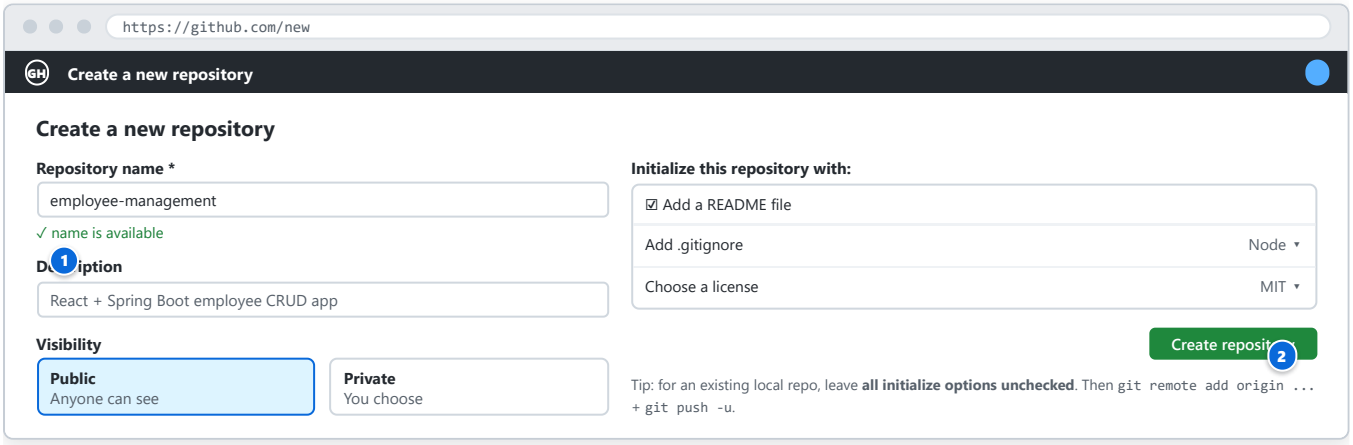


Fig 3.1 ILLUSTRATIVE MOCKUP **New repository page.** ① Name + visibility (public/private). ② Create button — select initialization options.

3.3 Public vs Private Repository

A common interview question. The free plan gives unlimited public and private repos; the difference is only in access.

Aspect	Public	Private
Who can view	World (anyone with link)	Only you + invited collaborators
Who can clone	Anyone	Members with access
Actions minutes	Free (standard runners)	Free quota, then billed
Pages hosting	Free	Paid plans (Pro/Team)
Best for	Portfolio, open source, learning	Company code, client work, secrets
Security	Everyone sees code + history	Controlled; still rotate secrets

GOLDEN RULE

Never commit `.env`, API keys, database passwords, or client data to a public repo. A private repo isn't safe either — always keep secrets in GitHub Secrets.

3.4 README.md — Repo Home Page

When someone opens your repo, the first thing they see is the README.md content. It is your project's **front page + documentation + recruiter pitch**.

README.md

```
# Employee Management System

React + Spring Boot CRUD application with JWT login.

## Tech Stack
React 18 · Spring Boot 3 · MySQL 8 · JWT
## Features
Employee CRUD, JWT authentication, role-based access.

## Run Locally
```bash
git clone https://github.com/dhamo-dev/employee-management.git
cd backend && mvn spring-boot:run
cd ../frontend && npm install && npm run dev
```

Author
Dhamodharan – [LinkedIn](https://linkedin.com/in/...)
```

Must have: project name + one-line purpose, tech stack, setup commands, live demo, screenshots, folder structure, author contact. **Nice to have:** build/license badges, API table, architecture diagram, roadmap checklist.

3.5 LICENSE — Legal Permission

Without a license, code is "all rights reserved" — nobody can legally use or copy it. Open source needs a license. Company internal repos usually don't need one.

License	You allow	Common use
MIT	Use, modify, sell — keep copyright notice	Most beginner/portfolio projects
Apache 2.0	Same as MIT + patent protection	Enterprise-friendly open source

3.6 .gitignore Templates

`When you create a repo`, GitHub generates a template automatically if you pick a language in the **Add .gitignore** dropdown. If not, create the file manually in the repo root.

Node
`node_modules/`
`dist/`
`.env`

Java
`target/`
`*.class`
`.idea/`

OS / IDE
`.DS_Store`
`Thumbs.db`
`.vscode/`

3.7 Repository Settings (Important Tabs)

Setting	What it controls
General	Repo name (rename), default branch, description, topics, archive/delete.
Collaborators	People + permission level (Read / Triage / Write / Maintain / Admin).
Branches	Branch protection rules: required reviews, status checks, no force-push.
Actions	Workflow permissions, secrets, runner access.
Pages	Static site hosting config (Chapter 11).
Danger Zone	Transfer ownership, archive (read-only), delete permanently.

DISCOVERY TIP · TOPICS
Under Settings → General → **Topics** add `react`, `spring-boot`, `jwt` — the project becomes discoverable in search.

3.8 Watchers, Stars & Forks

★ Star
Like/save. Works like a bookmark — your starred repos show on your profile. A popularity signal.

◎ Watch
Repo notifications. All activity, issues only, or ignore. Set company repos to issues-only.

⩾ Fork
A copy of someone else's repo in your account. The first step to open source contribution (Chapter 7).

Note: star/fork = independent copy; changes don't auto-sync. To contribute to the original repo you need a Pull Request (Chapter 7).

3.9 Repository Description & Social Preview

Fill in the description (one line) + website URL + topics. Upload a 1280×640 image under Settings → General → **Social preview** and LinkedIn/Twitter shares show a professional card.

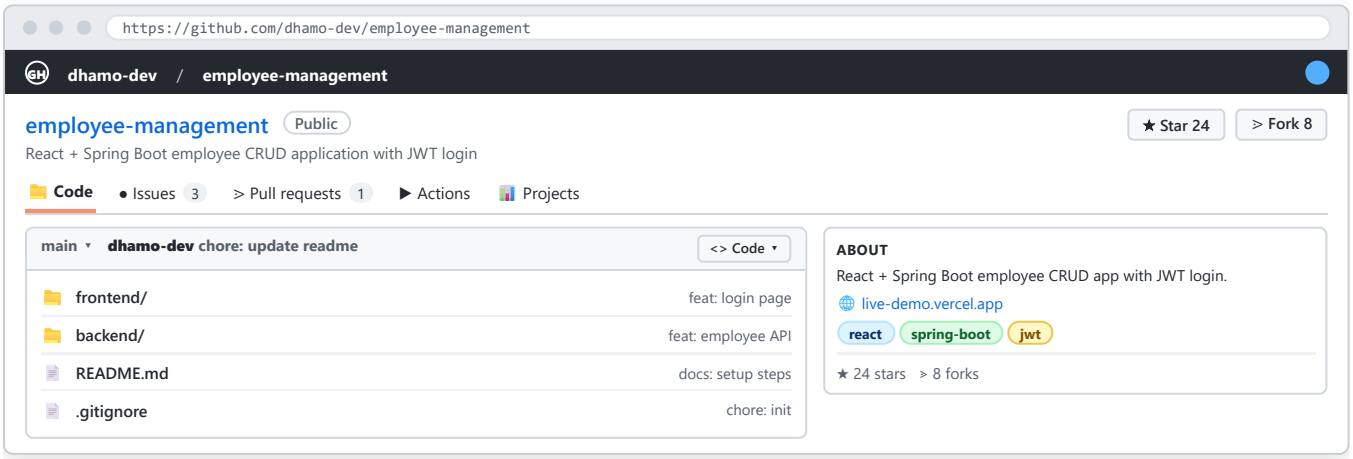


Fig 3.2 ILLUSTRATIVE MOCKUP **Repository home page.** README, topics, description, stars, and file list — recruiter's first impression.

3.10 Releases & Tags

We publish a deployment-ready version as a **release**. Internally that's a Git **tag** (version label) + release notes. For QA/Production we hand over a fixed version like "v1.2.0".

terminal

```
git tag -a v1.0.0 -m "First production release"
git push origin v1.0.0

# later releases
git tag -a v1.1.0 -m "Add employee search"
git push origin --tags
```

GitHub UI: repo → **Releases** → **Draft a new release** → select tag → title + release notes → attach build files (.jar, .zip) → Publish. List features, bug fixes, and breaking changes in the release notes.

Tag
Git-level pointer to a commit. `v1.0.0`

Release
GitHub-level package: tag + notes + binaries.

SemVer
`MAJOR.MINOR.PATCH` — 1.2.3 (Chapter 12).

3.11 Transfer, Archive & Delete

Action	What happens	When to use
Transfer	The repo moves to a new owner (user/org); URL changes; stars/forks/issues carry over.	Move a personal project to an org; change team ownership.
Archive	Becomes read-only. Issues/PRs/comments freeze; clone still works.	You won't maintain an old project; keep it for reference.
Delete	Repo + issues + PRs + releases are permanently deleted. 90-day recovery only via support (sometimes).	Clean up duplicate/dummy repos — confirm twice.

BEFORE DELETE
`Settings → General → Danger Zone`. Type the repo name to confirm. If you have a local clone the `.git` history remains, but GitHub issues/PRs/releases are gone. Don't delete a team repo — archive it.

REAL-TIME IT COMPANY USAGE
Company repo naming conventions are strict: `team-project-service` (e.g. `payments-invoice-api`). README mandatory sections: owner team, on-call contact, setup, deploy steps, and architecture link. Branch protection + CODEOWNERS + release tags all get configured when the repo is created. Recruiters/interviewers evaluate your personal repo by its README, commit frequency, and project structure.

3.12 Chapter Recap & Practice

- Repo name lowercase-hyphen; fill description + topics.
- Public = portfolio, Private = company code.
- README = setup steps + screenshots + tech stack.
- License (MIT) for open source; releases = tag + notes.

PRACTICE TASK
Create your first portfolio repo: README with 3 sections, MIT license, Node .gitignore, 5 topics, and one release `v0.1.0`. Then open repo Settings and note what each option does.

CHAPTER 04 · CONNECT TO GITHUB

Remote Repository

Connect local code to GitHub — push, pull, fetch, clone, and SSH setup.

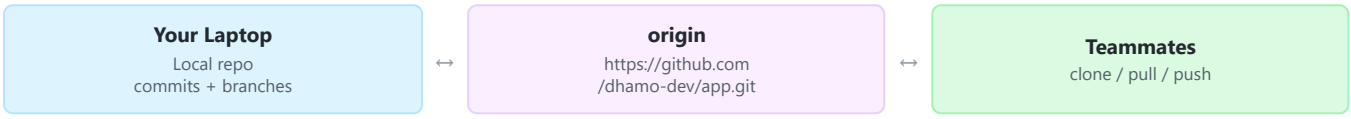
11 topics | Level: **Beginner** | Time: **75 min** | Needs: **GitHub repo**

In Chapter 3 we created an empty repo on GitHub. Now you need to connect the code on your laptop to it. That bridge is the **remote**. Without a remote there's no team collaboration, no backup, no CI/CD.

4.1 What is a Remote Repository?

DEFINITION

A remote repository is a copy of your local repo that lives on a server (GitHub). In Git a repo can have multiple remotes; the default name is `origin`. A remote = bookmarked URL + tracking branches.



Key idea: committed changes stay local only. Only `git push` sends them to GitHub. Teammates get them only when they run `git pull`. A remote does not sync automatically.

4.2 git remote add — Connect Local to GitHub

terminal

```
# copy the URL from the GitHub repo page
git remote add origin https://github.com/dhamo-dev/git-practice.git
git push -u origin main
```

`origin` = the nickname we give the remote (any name is allowed, but origin is convention). `-u` = sets upstream; after that `git push` alone is enough. If you have an **existing repo with commits**, don't initialize with README/gitignore when creating the GitHub repo — you'll get a conflict.

4.3 git remote -v / rename / remove

terminal

```
git remote -v                        # list remotes with URLs
git remote show origin               # branches + tracking details
git remote rename origin upstream    # rename
git remote set-url origin git@github.com:dhamo-dev/app.git
git remote remove origin             # disconnect (local repo stays)
```

Git Bash — git remote -v

```
$ git remote -v
origin  https://github.com/dhamo-dev/git-practice.git (fetch)
origin  https://github.com/dhamo-dev/git-practice.git (push)
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/dhamo-dev/git-practice.git
  HEAD branch: main
```

Common use: `set-url` after a repo transfer; in a fork workflow `origin` = your fork, `upstream` = original repo (Chapter 7).

4.4 git push — Upload Commits

Sends local commits to the remote. Running `git pull` before pushing avoids conflicts (a must in team repos).

terminal

```
git push                            # after upstream is set
git push -u origin main             # first push
git push origin feature/login       # specific branch
git push --force-with-lease         # safe force (never on main)
git push origin --delete old-branch # delete remote branch
```

Git Bash — first push output

```
$ git push -u origin main
Enumerating objects: 24, done.
Writing objects: 100% (24/24), 6.42 KiB | 1.20 MiB/s, done.
To https://github.com/dhamo-dev/git-practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

NEVER FORCE-PUSH SHARED BRANCHES

`git push --force` overwrites remote history; teammates may lose commits. Force push on main/develop is totally banned. If it's unavoidable, use only `--force-with-lease`.

4.5 git pull — Download + Merge

`git pull` = `git fetch` + `git merge`. It downloads remote changes and merges them into the current branch. Do this before your daily standup.

terminal

```
git pull                    # fetch + merge (default)
git pull --rebase           # fetch + rebase (clean linear history)
git pull origin develop     # specific branch
```

Git Bash — git pull

```
$ git pull
remote: Enumerating objects: 7, done.
From https://github.com/dhamo-dev/git-practice
   4b8e2f1..9c1d4ab  main       -> origin/main
Updating 4b8e2f1..9c1d4ab
Fast-forward
 src/App.jsx | 12 ++++++------
 1 file changed, 6 insertions(+), 6 deletions(-)
```

PULL REJECTED?

If you get `error: Your local changes would be overwritten by merge`, commit your local changes or run `git stash`. Then `git pull`. If there's a conflict, run `git status` and fix the files (Chapter 5).

4.6 git fetch — Download Only

`fetch` doesn't touch your working files. Only the `origin/main` pointer updates. It's the best way to safely see what's coming, then decide.

terminal

```
git fetch                # origin updates download, no merge
git fetch origin main    # one branch
git fetch --all --prune  # all remotes, delete stale refs
git log HEAD..origin/main --oneline   # what's new?
```

Git Bash — review before merge

```
$ git fetch origin
From https://github.com/dhamo-dev/employee-management
   7f3a91c..4b8e2f1  main -> origin/main
$ git log HEAD..origin/main --oneline
4b8e2f1 feat: add login form validation
1a7c002 feat: create login page UI
$ git merge origin/main
Updating 7f3a91c..4b8e2f1
Fast-forward
```

4.7 git fetch vs git pull

Point	git fetch	git pull
What it does	Downloads remote commits only	Downloads + merges into current branch
Working files	Not touched	Changed immediately
Merge conflicts	No (nothing merged)	Possible
Safe?	Always safe	Depends on local changes
Use when	Review first, then merge manually	You want latest code now

4.7.1 The safe sync workflow (recommended)

1. **Fetch first**
   `git fetch origin` — see what's coming, files stay safe.
2. **Review incoming changes**
   `git log HEAD..origin/main --oneline` and `git diff HEAD origin/main`.
3. **Decide**
   If nothing looks wrong, `git merge origin/main` (or `git pull`). If you have local changes, commit/stash them.

Thing	Syncs automatically?	How
Commits	No	`git push` / `git pull`
Branches	No	`git push -u origin branch`
Tags	No	`git push --tags` / `git fetch --tags`
Stashes	Never	Local only — never goes to the remote
Untracked files	Never	Only shared when committed
Remote URL config	No	`git remote set-url`

4.8 git clone — Copy a Repository

Copies the remote repo in full (history, branches, tags and all) to your laptop. On your first day at a company, this is the first command.

terminal

```
git clone https://github.com/dhamo-dev/employee-management.git
git clone git@github.com:dhamo-dev/employee-management.git   # SSH
```

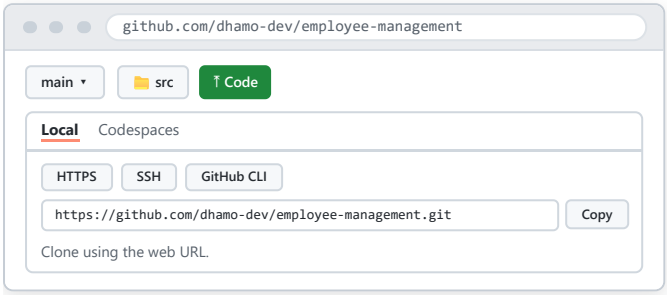


Fig 4.1 ILLUSTRATIVE MOCKUP **Code button → clone URL.** Pick the HTTPS or SSH tab and copy the URL.

Git Bash — clone output

```
$ git clone https://github.com/dhamo-dev/employee-management.git
Cloning into 'employee-management'...
Receiving objects: 100% (186/186), 42.10 KiB
Resolving deltas: 100% (98/98), done.
$ cd employee-management
$ git remote -v
origin  https://github.com/dhamo-dev/employee-management.git
```

Clone vs ZIP: ZIP has no history, no branches, and can't push. Always clone.

4.9 Upstream Tracking Branches

`-u` (or `--set-upstream`) sets a remote partner for the branch. After that you don't need to type the remote/branch name in `git push` / `git pull` commands.

terminal

```
git push -u origin feature/login
git branch -vv                # tracking info (ahead/behind)
git branch --set-upstream-to=origin/main main
```

Git Bash — git branch -vv

```
$ git branch -vv
* main          9c1d4ab [origin/main] merge: feature/login
  feature/login 4b8e2f1 [origin/feature/login: ahead 2] login form
```

Read it: `ahead 2` = 2 local commits pending push. `behind 3` = 3 remote commits pending pull. `ahead 1, behind 2` = diverged — resolve with pull/rebase.

AFTER CLONE — FIRST THREE COMMANDS

`git log --oneline -5` → recent history · `git branch -a` → local + remote branches · `git remote -v` → verify origin URL. Then run `git switch -c feature/my-task` and start working.

4.10 SSH vs HTTPS Authentication

You must prove your identity when pushing/pulling to GitHub. There are two options — HTTPS is easy for beginners, SSH is better for daily developers.

Point	HTTPS	SSH
URL format	`https://github.com/user/repo.git`	`git@github.com:user/repo.git`
Auth method	Personal Access Token (PAT)	SSH key pair
Setup	Easy — generate a token	Moderate — keygen + add to GitHub
Password prompt	Every push (without a helper)	No prompt after setup
Firewall/proxy	Works everywhere (port 443)	Port 22 may be blocked
Best for	Beginners, CI, demo laptops	Daily development, multiple repos

SSH setup (one time)

```
ssh-keygen -t ed25519 -C "dhamo@example.com"
# Enter x3 (default path, empty passphrase)
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
# GitHub → Settings → SSH keys → New
ssh -T git@github.com
```

HTTPS TOKEN NOTE

You can't use your GitHub password as a push password (removed in 2021). Create a PAT: Settings → Developer settings → Personal access tokens → Fine-grained token. Paste the token as the password. **Token = password** — don't share it.

Git Bash — SSH test

```
$ ssh -T git@github.com
Hi dhamo-dev! You've successfully authenticated...
```

4.11 Credential Cache & Troubleshooting

terminal

```
# remember HTTPS credentials
git config --global credential.helper manager   # Windows
git config --global credential.helper cache     # 15 min memory
```

Error	Fix
`Password authentication was removed`	Use a PAT, not your password.
`Permission denied (publickey)`	SSH key not added to GitHub; test with `ssh -T`.
`remote origin already exists`	`git remote set-url origin <url>`.
`failed to push some refs`	New commits on the remote; `git pull --rebase` then push.

REAL-TIME IT COMPANY USAGE

SSH keys are mandatory in companies; one key pair, add the public key. Revoke as soon as someone is offboarded. CI/CD uses OIDC — avoid long-lived PATs.

4.12 Chapter Recap & Practice

Practice: create an empty repo → connect locally → `git push -u origin main` → edit & commit on GitHub → run `git log HEAD..origin/main`.

CHAPTER 05 · PARALLEL DEVELOPMENT

Branching and Merging

Create a branch, work in parallel, merge it back, and resolve conflicts with confidence.

12 topics | Level: **Core skill** | Time: **90 min** | Most asked: **Interviews**

This is Git's real power. A branch is an independent timeline: you can build a feature without touching main. In a company, **never commit directly to main** — all work goes through a feature branch and a PR.

5.1 What is a Branch?

A branch is a movable pointer to a commit. The `main` branch points to the latest commit. Creating a new branch does not copy files — Git just creates a 41-byte pointer file. That is why branching is cheap and instant.

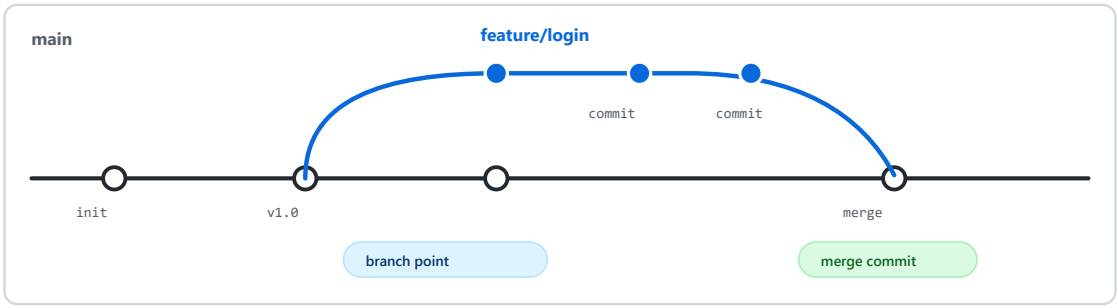


Fig 5.1 Branch = parallel timeline. From the branch point, feature commits move separately; at the merge they join main again.

5.2 git branch — Create & List

terminal

```
git branch                    # list local branches
git branch -a                 # + remote branches
git branch feature/login      # create (stay on current)
git branch -d feature/login   # delete (merged only)
git branch -D feature/login   # force delete (unmerged)
```

Git Bash — git branch

```
$ git branch
* main
  feature/login
  fix/navbar
```

5.3 git switch / git checkout

To create a branch and move to it, use `git switch -c` (modern, recommended) or `git checkout -b` (old but everywhere).

terminal

```
git switch main                  # move to main
git switch -c feature/login      # create + move
git switch -                     # previous branch
git checkout -b hotfix/payment   # old style, same result
git checkout 4b8e2f1             # detached HEAD (view old commit)
```

UNCOMMITTED CHANGES?

You can't switch: `error: Your local changes would be overwritten`. Commit them or run `git stash`. When you switch branches, uncommitted changes follow you — so there is a chance you commit to the wrong branch by accident.

5.4 Delete & Rename Branches

local branches

```
git branch -d feature/login     # safe delete
git branch -D feature/old       # force
git branch -m feature/auth      # rename current
git push origin --delete feature/login
```

RULE

Delete only branches that are already merged. If `-d` refuses, there are unmerged commits — verify with `git log main..feature/login`. Deleting the remote removes the branch on GitHub, but the PR history stays.

5.5 Merge — Fast-Forward

If main has no new commits after the feature branch was created, Git just moves the pointer forward during the merge. No new merge commit is created. History stays a straight line.

terminal

```
git switch main
git merge feature/login          # fast-forward if possible
git merge --no-ff feature/login  # always create merge commit
git merge --abort                # cancel a conflicted merge
```

Git Bash — fast-forward merge

```
$ git switch main
Switched to branch 'main'
$ git merge feature/login
Updating 7f3a91c..4b8e2f1
Fast-forward
 src/Login.jsx | 48 ++++++++++++++++++++
 1 file changed, 48 insertions(+)
```

5.6 Three-Way Merge

When main and the feature branch have both advanced with different commits, Git uses the common ancestor plus the two branch tips (three points) to create a merge commit. This is a **recursive / ort merge**.

Common ancestor 7f3a91c → **main tip** 9c1d4ab + **feature tip** 4b8e2f1 → **Merge commit** new commit, 2 parents

terminal

```
git switch main
git pull                         # latest main
git merge feature/login
# output: Merge made by the 'ort' strategy.
```

Merge type	When	History
Fast-forward	No new commits on main	Linear, no merge commit
Three-way	Both branches advanced	Merge commit with 2 parents
Squash	Many small commits → one	Linear, detailed history lost
Rebase	Replay commits on new base	Linear, no merge commit

5.7 What is a Merge Conflict?

When two branches change **the same lines in the same file** differently, Git can't decide, so it marks a conflict. This is not an error — it happens daily in normal team workflow, and you decide the final code.

Git Bash — conflict output

```
$ git merge feature/login
Auto-merging src/App.jsx
CONFLICT (content): Merge conflict in src/App.jsx
Automatic merge failed; fix conflicts and then commit the result.
```

When conflicts happen
Same line edited in both branches · file deleted in one, edited in the other · rename + modify · binary files changed.

When they don't
Different files · different lines in the same file · one branch only added new content below.

5.8 Resolve a Conflict — Step by Step

1. **Check status**
   `git status` shows `both modified: src/App.jsx`. Look at the `Unmerged paths` list.

2. **Open the file**
   In VS Code the markers appear: `<<<<<<< HEAD`, `=======`, `>>>>>>> feature/login`. Inline buttons: **Accept Current**, **Incoming**, **Both**.

3. **Choose the final code**
   If you want both, combine them manually. **Fully remove** the markers, then test.

4. **Stage & commit**
   `git add src/App.jsx` → `git commit`. Git suggests a default merge message; you can add a meaningful one.

5. **Verify**
   `git log --oneline --graph -5` and run the app to check functionality. Then delete the branch.

src/App.jsx — conflict markers

```
<<<<<<< HEAD
  <h1>Welcome back</h1>
=======
  <h1>Welcome, {user.name}</h1>
>>>>>>> feature/login
  <LoginForm />   // code continues
```

GOOD RESOLUTION
Merge both intentions: `<h1>Welcome back, {user.name}</h1>`. Commit only when the markers are gone, the app runs, and tests pass.

NEVER DO THIS
Never commit with conflict markers left in the code. Never blindly use `--ours` / `--theirs` — the other side's work gets lost.

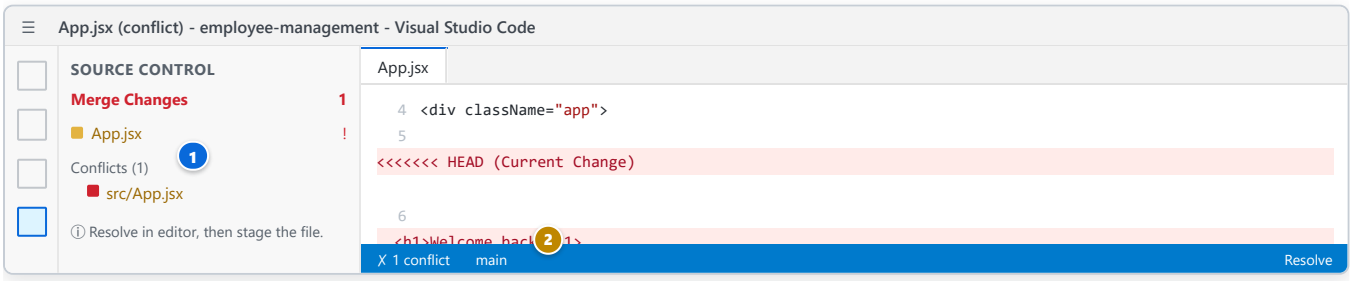


Fig 5.2 ILLUSTRATIVE MOCKUP **VS Code merge conflict view.** ① Conflicted file in Source Control. ② Current vs Incoming changes — use the Accept buttons to select the final code.

5.9 git rebase — Move Commits to a New Base

It replays your feature branch commits on top of main's latest commit. Result: straight-line history, no merge commit. PR review stays clean.

terminal

```
git switch feature/login
git fetch origin
git rebase origin/main
# if a conflict appears
git add <file>
git rebase --continue
git rebase --abort          # cancel and go back
```

GOLDEN RULE OF REBASE

Never rebase shared/public branches. Rebase rewrites commit hashes, so teammates' history no longer matches. Rebase only **your own feature branch**. When you push you need `--force-with-lease` (a normal push is rejected).

5.10 git merge vs git rebase

Point	Merge	Rebase
History	Preserves real branching + merge commits	Rewrites into a linear history
Commit hashes	Unchanged	New hashes (rewritten)
Conflicts	Resolve once	Resolve per replayed commit
Safe on shared branch?	Yes	No — never
Best for	Merging PRs into main	Updating your feature branch

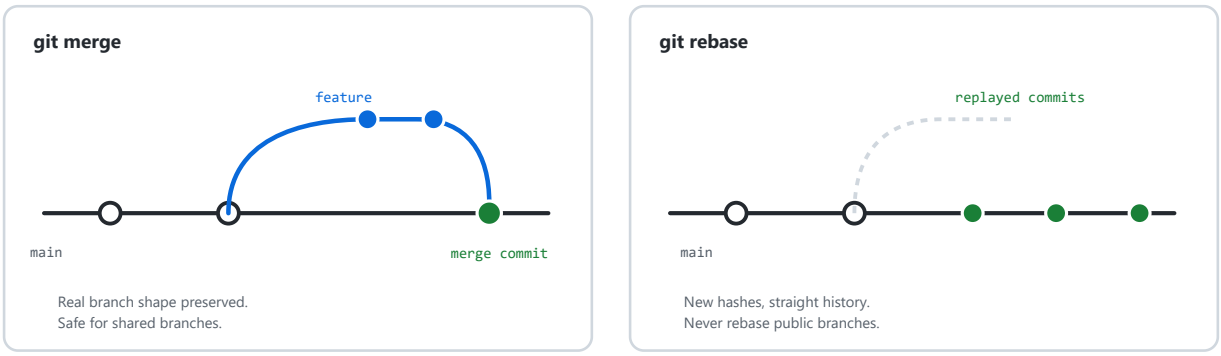


Fig 5.3 **Same work, two histories.** Merge keeps the branch shape and adds a merge commit; rebase replays your commits on top of main for a clean, linear log.

5.11 git cherry-pick — Copy One Commit

Copies a specific commit from one branch onto the current branch. Useful when a hotfix is needed on main but not on the feature branch.

terminal

```
git switch main
git log --oneline feature/payment | head -5
git cherry-pick 8f2c1ab
# if a conflict appears: fix → git add → git cherry-pick --continue
```

Git Bash cherry-pick

```
$ git cherry-pick 8f2c1ab
[main 3c9e7d2] fix: validate payment amount
 Date: Mon Nov 9 10:42:11 2026 +0530
 1 file changed, 4 insertions(+), 1 deletion(-)
```

Note: cherry-pick creates a duplicate commit (same change, new hash). Over time the branches diverge, so use it only for emergency fixes.

5.12 Branching Strategies

Strategy	How it works	Best for
GitHub Flow	main + short-lived feature branches + PR + deploy	Web apps, startups, CI/CD teams
Git Flow	main, develop, feature/*, release/*, hotfix/*	Versioned releases, enterprise
Trunk-based	Everyone commits to main frequently, tiny branches	High-performing DevOps teams
Release branch	release/1.2 cut, only bug fixes, then merge to main	Mobile apps, quarterly releases

Team Branch naming and lifecycle rules

Naming convention

`feature/login-page` · `fix/navbar-overlap` · `hotfix/payment-500` · `chore/update-deps`. Lowercase, hyphen-separated, ticket ID optional: `feature/EMS-142-login`.

Lifecycle

Create from updated `main` → commit small → push daily → open a draft PR early → review → merge → delete the branch. Keep a branch alive for under 1–3 days.

COMMON MISTAKES

Long-lived branches (merge pain grows daily) · branching from stale main · direct commits to main · deleting remote branches before the PR merge · rebasing a branch that teammates already pulled.

REAL-TIME IT COMPANY USAGE

Most product teams follow GitHub Flow: `main` is always deployable, a feature branch lives 1–3 days max, PR review needs at least one approval, and merging deploys automatically through CI. Git Flow is still used in banking/enterprise where there is a monthly release.

Practice: Create 2 commits on main and 2 commits on a feature branch. Try a fast-forward merge, then a three-way merge (with a parallel commit on main). Edit the same line in the same file to create a conflict, then resolve it.

CHAPTER 06 · SAFE RECOVERY

Advanced Git

Undo mistakes, park unfinished work, and tag releases — the tools professional developers use.

16 topics | Level: **Intermediate** | Time: **120 min** | Most asked: **Scenarios**

This is the difference between a beginner and a senior. You committed the wrong code, you need to clean up a branch, you need to tag a version for production — this chapter covers all of it step by step. You can practice everything **safely in a local repo**.

6.1 git stash — Park Unfinished Work

Set half-done code aside temporarily without committing it. When an urgent hotfix arrives, stash the current feature, switch to main, fix it, then apply the stash again. **Stash is local-only** — it isn't pushed and teammates don't see it.

terminal

```
git stash                          # save tracked changes
git stash push -m "login wip"      # with a message
git stash -u                       # include untracked files
git stash list                     # see all stashes
git stash show -p stash@{0}        # inspect diff
```

Git Bash — stash

```
$ git stash push -m "login wip"
Saved working directory and index state On main: login wip
$ git stash list
stash@{0}: On main: login wip
```

6.2 stash apply / pop / drop

terminal

```
git stash apply               # restore, keep in stash list
git stash pop                 # restore and remove from list
git stash apply stash@{2}     # pick a specific stash
git stash drop stash@{0}      # delete one
git stash clear               # delete all (careful)
```

COMMON MISTAKE

`stash pop` does not delete the stash automatically when there is a conflict — resolve it, then run `git stash drop`. Don't think of a stash as a backup: deleting or rebasing a branch gets confusing. For long-term work, a WIP commit or draft PR is better.

STASH VS WIP COMMIT

Stash = private, temporary, no history. WIP commit = visible on the branch, can be pushed, teammates can review it. Use stash for one hour of work; a WIP commit + draft PR is better for multi-day work.

6.3 git reset — Move HEAD Back

`reset` moves the branch pointer to an older commit. The **mode** decides how many changes to keep. On a shared branch this rewrites history — don't use `reset` on commits that are already pushed; use `revert` instead.

Mode	HEAD moves	What happens to changes
`--soft`	Yes	Keep staged (ready to commit again)
`--mixed` (default)	Yes	Keep in working dir, unstaged
`--hard`	Yes	**Discard everything** — dangerous

terminal

```
git reset --soft HEAD~1      # undo commit, keep files staged
git reset HEAD~1             # undo commit + unstage
git reset --hard HEAD~1      # throw away last commit + changes
git reset --hard origin/main # match remote exactly
```

DANGER ZONE

`--hard` permanently deletes uncommitted work. Run `git stash` first or note the hash from `git reflog`. Reflog helps recover an unpushed commit; it does not help with untracked files.

6.4 git revert — Safe Undo on Shared Branch

Instead of deleting an old commit, it creates a **new commit that does the opposite**. History is preserved, so it is safe on the main branch. This is the correct way to undo a production bug commit.

terminal

```
git revert 8f2c1ab             # revert one commit
git revert HEAD                # revert latest
git revert 8f2c1ab..3c9e7d2     # range (careful with order)
git revert --no-edit 8f2c1ab    # accept default message
```

Git Bash revert

```
$ git revert --no-edit 8f2c1ab
[main 7d1a0c4] Revert "fix: validate payment amount"
 1 file changed, 1 insertion(+), 4 deletions(-)
```

Map Which undo command should I use?

Situation	Correct command
Uncommitted file change	`git restore file.js`
Staged but not committed	`git restore --staged file.js`
Last commit, not pushed	`git reset --soft HEAD~1`
Commit already pushed	`git revert <hash>`

6.5 git reflog — Your Safety Net

Reflog records every action where HEAD moved: commit, reset, rebase, merge, checkout. If you make a mistake you can recover for up to 30 days (default). **Nothing in Git is truly lost** — the problem is not knowing about reflog.

terminal

```
git reflog                    # every HEAD movement
git reflog show feature/login  # branch-specific
git reset --hard HEAD@{3}      # go back to entry 3
git branch rescue HEAD@{5}     # create branch at old state
```

Git Bash — git reflog

```
$ git reflog
7d1a0c4 HEAD@{0}: reset: moving to HEAD~1
8f2c1ab HEAD@{1}: commit: fix: validate payment
4b8e2f1 HEAD@{2}: checkout: moving from main to feature/login
```

RECOVERY RECIPE

Ran the wrong `reset --hard`? Open `git reflog`, find the lost commit hash, then run `git reset --hard <hash>` or `git branch rescue <hash>`. Check reflog first — no need to panic.

6.6 git tag — Mark a Release

Give a commit a permanent name to mark a release: `v1.0.0`, `v1.2.3`. A branch moves, a tag does not. In CI/CD a tag push triggers an automatic production deploy.

terminal

```
git tag v1.0.0                          # lightweight
git tag -a v1.0.0 -m "First stable release" # annotated
git tag                                 # list local tags
git show v1.0.0                         # tag details
git push origin v1.0.0                  # push one tag
git push origin --tags                  # push all tags
git tag -d v1.0.0                       # delete local tag
```

SEMVER REMINDER

`MAJOR.MINOR.PATCH` → breaking change / new feature / bug fix. Always prefer an annotated tag (`-a`) — it stores the tagger name, date, and message. To delete a tag you need a separate `git push origin --delete v1.0.0` on the remote.

1. **Tag the release commit**
   `git tag -a v1.4.0 -m "Release 1.4.0"` — tag the exact commit that was merged into main.
2. **Push the tag**
   `git push origin v1.4.0`. A tag push triggers the CI/CD production deploy.
3. **Create GitHub Release**
   Repo → Releases → Draft a new release → tag select → auto-generated notes → Publish.
4. **Attach artifacts**
   Attach the build zip, checksum, and migration notes. The release page is the official changelog for customers.

6.7 git commit --amend — Fix the Last Commit

Use it to merge a typo in the last commit message, a forgotten file, or a small fix into the same commit. It creates a **new hash** — amending an already pushed commit needs a force push, so avoid it on shared branches.

terminal

```
git add forgotten-file.js
git commit --amend --no-edit      # keep same message
git commit --amend -m "feat: add login validation"
git commit --amend --author="Dhamo <dhamo@example.com>"
```

ONLY BEFORE PUSH

During PR review, add a new commit for a "small fix" — amend + force push only confuses the reviewer. Use amend only on your own feature branch. After amend: `git push --force-with-lease`.

6.8 Interactive Rebase — Clean Your History

Before pushing you can reorder, reword, squash, or drop commits. A clean history in the PR is easy for the reviewer. `git rebase -i HEAD~3` opens the last 3 commits.

terminal

```
git rebase -i HEAD~3
# editor opens:
# pick   8f2c1ab feat: login form
# squash 3c9e7d2 fix typo
# reword 7d1a0c4 add validation
git rebase --continue
git rebase --abort
```

Command	Meaning
pick	Keep commit as-is
reword	Change commit message
squash	Merge into previous commit, edit message
fixup	Merge into previous, discard message
drop	Delete commit

6.9 Squash Merge

Even if the feature branch has 12 small commits, you can create **one clean commit** on main. Select "Squash and merge" in the PR merge button, or in the terminal:

terminal

```
git switch main
git merge --squash feature/login
git commit -m "feat: add login page (#42)"
```

6.10 git worktree — Two Branches at Once

Multiple folders in the same repo, each checking out a different branch. To handle an urgent hotfix you don't need to stash your current work — create a separate worktree. Build files and node_modules stay separate.

terminal

```
git worktree add ../hotfix-folder hotfix/payment
git worktree list
cd ../hotfix-folder
# fix, commit, push normally
git worktree remove ../hotfix-folder
git worktree prune
```

6.11 Git Hooks — Automate Checks

A script runs automatically when a commit or push happens. Samples are in the `.git/hooks/` folder. To share them with the team, people use **Husky** (Node) or the pre-commit framework, because `.git/hooks` is not pushed.

Hook	When it runs
pre-commit	Before commit created — lint, format, tests
commit-msg	Validate message format (Conventional Commits)
pre-push	Before push — unit tests, secret scan
post-merge	After merge — npm install, DB migrate

.husky/pre-commit

```
#!/bin/sh
npx lint-staged
npm test -- --watchAll=false
```

terminal

```
npm install --save-dev husky lint-staged
npx husky init
echo "npx lint-staged" > .husky/pre-commit
```

COMMON MISTAKE

You can bypass hooks with `git commit --no-verify` — but the same checks must be mandatory in CI. Hooks cannot replace CI checks; you need both.

WORKTREE IN REAL LIFE

When a production hotfix arrives: your main worktree has feature code, so check out a clean main in the `../hotfix` worktree and fix it there. You can open both in two IDE windows — same repo, different branches.

6.12 Git Submodules

Include one repo as a dependency of another. A submodule is a pointer to a specific commit in a separate repo. Useful for sharing a library without a monorepo, but updating and versioning are complex.

terminal

```
git submodule add https://github.com/org/ui-kit.git libs/ui-kit
git submodule update --init --recursive   # after clone
git submodule update --remote             # latest remote
git submodule status
```

ALTERNATIVE

Modern teams prefer a package manager: an npm private package, a Maven artifact, or a monorepo. If you use submodules, commit `.gitmodules` and clone with `--recursive` in CI.

Option	Best for
Submodule	Exact commit pinning, separate release cycle
Subtree	Code copied into repo, simpler clone for juniors
Package	Versioned library via npm/Maven — preferred
Monorepo	Many packages, one CI, shared tooling

6.13 git bisect — Find the Bad Commit

Don't know when the bug appeared? Use binary search to find the culprit commit in `log2(n)` steps. 1000 commits = about 10 checks. Manual or automated.

terminal

```
git bisect start
git bisect bad                # current version broken
git bisect good v1.2.0        # last known working tag
# test each checkout, then:
git bisect good   # or git bisect bad
git bisect reset              # back to original branch
```

Git Bash bisect

```
$ git bisect good
Bisecting: 6 revisions left to test after this (roughly 3 steps)
[a1b2c3d] feat: update checkout API
```

terminal

```
git bisect start
git bisect bad HEAD
git bisect good v1.2.0
git bisect run npm test -- --watchAll=false
git bisect reset
```

Automate: `git bisect run` — a passing test is marked good, a failing test is marked bad automatically. Exit code 125 = skip (flaky test).

6.14 git gc & prune — Clean the Repo

Git garbage-collects automatically, but you can force it manually. Unreachable objects are compressed or deleted. Useful for repos that take up a lot of disk space.

terminal

```
git gc                  # pack objects, clean garbage
git gc --aggressive     # slower, deeper cleanup
git prune               # delete unreachable objects
git count-objects -vH   # repo size summary
```

CAREFUL

`prune` permanently deletes unreachable commits — the chance of reflog recovery drops. `gc` is safe; `prune --expire now` is dangerously aggressive. For a repo with a large binary history, Git LFS migrate is better.

6.15 Git LFS — Large Files

Committing images, videos, PSDs, or datasets directly into the repo makes its size explode. Git LFS (Large File Storage) keeps the actual file in separate storage, and only a pointer stays in the repo.

terminal

```
git lfs install
git lfs track "*.psd"
git add .gitattributes design.psd
git commit -m "chore: track design files with LFS"
git lfs ls-files
```

COST NOTE

The GitHub free plan has LFS storage and bandwidth limits. To move an already committed large file to LFS, use `git lfs migrate import --include="*.psd"` — it rewrites history, so team coordination is needed.

6.16 .gitattributes — Per-File Rules

Define rules per file type: line endings, diff behavior, merge strategy, LFS tracking. This solves CRLF/LF issues in a Windows + Mac team.

.gitattributes

```
* text=auto
*.sh text eol=lf
*.bat text eol=crlf
*.png filter=lfs diff=lfs merge=lfs -text
package-lock.json merge=binary
```

REAL-TIME IT COMPANY USAGE

In product teams `.gitattributes` is mandatory: if Windows developers commit CRLF, shell scripts fail on Linux CI. `text=auto` + `eol=lf` is the standard. Design teams use LFS; legal/audit teams maintain tag-based releases.

Practice: Create a stash → apply → pop. Try `reset --soft` and `--mixed`. Recover a commit using `reflog`. Create an annotated tag and push it to the remote.

CHAPTER 07 · TEAM COLLABORATION

Pull Requests & Collaboration

Fork, branch, PR, review, merge — the official route for code to reach the team in a company.

13 topics | Level: **Core team skill** | Time: **100 min** | Interviews: **Very high**

In a company, **you never push directly to the main branch**. Every change follows branch → Pull Request → review → CI → merge. Without this flow, knowing Git is not enough to work in a team. That is the main goal of this chapter.

7.1 Fork — Your Own Copy

A fork is a complete copy of someone else's repo in your account. It is the first step for contributing to open source. You have **no write access** to the original repo — you work in the fork and suggest changes through a PR. For internal company repos, forks are usually unnecessary; a branch is enough.

terminal

```
# GitHub UI: repo → Fork → Create fork
git clone https://github.com/your-name/react.git
cd react
git remote add upstream https://github.com/facebook/react.git
git remote -v
```

ORIGIN VS UPSTREAM

`origin` = your fork (push access). `upstream` = the original project (read-only). Commit to the fork, then open a PR against upstream.

7.2 Sync Your Fork

When upstream gets new commits, your fork falls behind. Sync regularly to keep the PR clean. The GitHub UI has a **Sync fork** button; in the terminal:

terminal

```
git fetch upstream
git switch main
git merge upstream/main        # or git rebase upstream/main
git push origin main
```

COMMON MISTAKE

If you work for months without syncing the fork, the PR will show hundreds of unrelated commits. Always run `git fetch upstream` before creating a feature branch.

7.3 Pull Request — What & Why

A PR is a request: "merge the changes in this branch into main". Code review, discussion, CI checks, approval, and audit history all happen in the PR. A PR is a **quality gate**, not just a merge button.

1. **Branch ready**
   Push the feature branch: `git push -u origin feature/employee-search`.
2. **Open PR**
   GitHub → Pull requests → New pull request → base `main`, compare your branch.
3. **Describe**
   Clear title, what/why, screenshots, test steps, linked issue (`Closes #128`).
4. **Review + CI**
   Assign a reviewer, wait for checks to pass, and fix the comments.
5. **Merge + cleanup**
   After approval and green checks, merge and delete the branch.

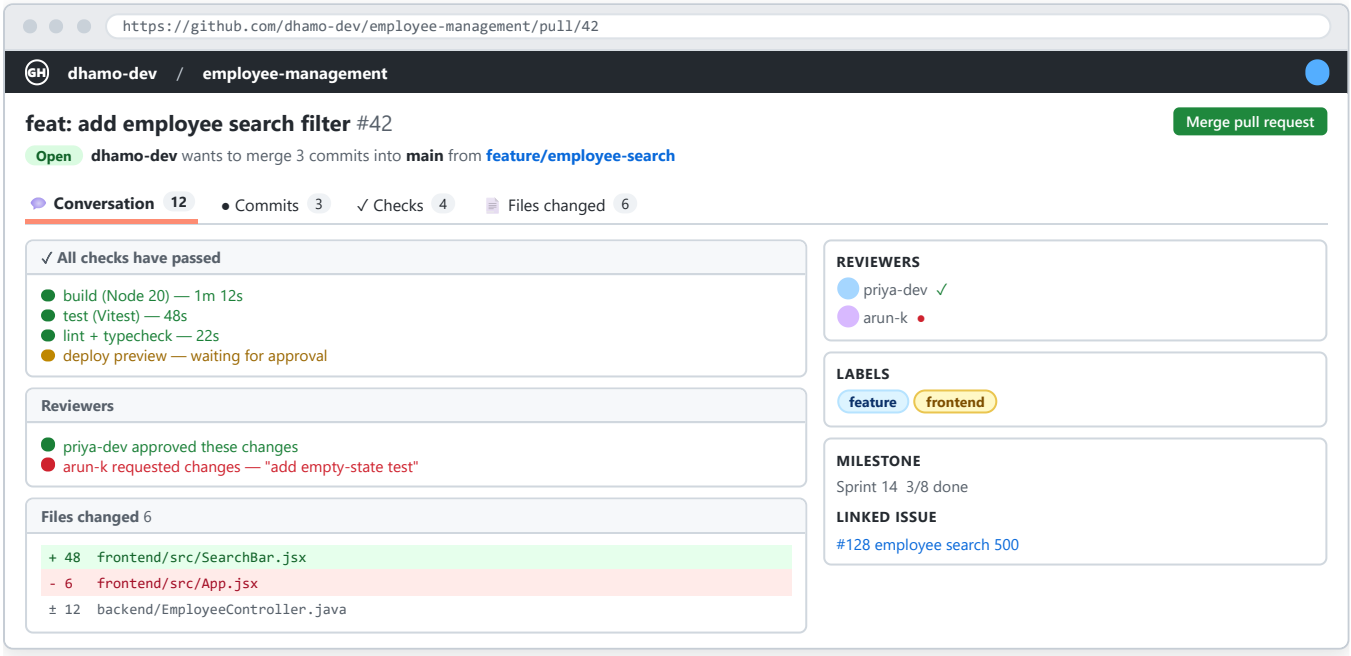


Fig 7.1 ILLUSTRATIVE MOCKUP **Pull request page.** Title, checks, reviewers, diff stats, labels, and linked issue — full context on one screen.

7.4 PR Template

Add `.github/PULL_REQUEST_TEMPLATE.md` to the repo and every new PR auto-fills the same checklist. Simple, but powerful for team consistency.

.github/PULL_REQUEST_TEMPLATE.md

```
## What changed
Employee search API + frontend filter bar.

## Why
Closes #128. Search was returning 500 for empty query.

## How tested
- npm test (12 passed)
- manual: empty, special chars, 500 rows

## Checklist
- [x] Tests added
- [x] No secrets committed
- [ ] Docs updated
```

PR TITLE MATTERS

`feat: add employee search` — good. `changes`, `fix`, `final code` — bad. With squash merge, the PR title becomes the main commit message (Chapter 12 convention).

7.5 Code Review — Give & Receive

As author

Small PR (<400 lines) · self-review first · attach screenshots · reply to reviewer questions · resolve comments. The "Nit:" prefix makes it clear a suggestion is non-blocking.

As reviewer

Check logic, security, tests, naming, and edge cases. Let the formatter handle style. **Approve** / **Request changes** / **Comment** — three options. A blocking comment needs a reason and a suggestion.

Review comment example — actionable

```
nit: extract this magic number into a constant
suggestion: if (!query.trim()) return [];
question: is 500 expected for an empty query?
```

7.6 Suggested Changes

A reviewer can propose a line-level edit directly in the code. If the author clicks **Commit suggestion**, GitHub creates the commit. Very fast for small fixes; logic changes still need an explanation.

GitHub review suggestion

```
- const results = employees.filter(e => e.name.includes(q));
+ const results = employees.filter(e =>
+   e.name.toLowerCase().includes(q.trim().toLowerCase())
+ );
```

7.7 Merge Strategies

Strategy	Result	Use when
Merge commit	All commits + merge commit preserved	Feature history valuable, release branch
Squash & merge	One clean commit on main	Messy WIP commits, most product teams
Rebase & merge	Linear history, individual commits kept	Every commit clean, trunk-based team

TEAM RULE

Enforce one strategy in repo settings. Mixing them makes history unpredictable. If you choose squash, follow the PR title convention strictly — it becomes the final commit message.

7.8 Draft Pull Request

Open a draft PR to share work in progress early. CI runs, reviewers get context, but the **merge button is disabled**. Click "Ready for review" when it is done.

terminal

```
git push -u origin feature/employee-search
gh pr create --draft --title "feat: employee search" --body "WIP"
gh pr ready 42
```

WHY DRAFT?

You can discuss architecture decisions early and avoid duplicate work. Mention "draft PR #42 is open" in the daily standup — transparency.

7.9 CODEOWNERS

Assign reviewers automatically by file path. Create `.github/CODEOWNERS` and enable "Require review from Code Owners" in branch protection. Then a backend change must be reviewed by the backend lead.

.github/CODEOWNERS

```
# default owners
*                   @dhamo-dev

# frontend
/frontend/          @priya-dev @arun-k

# backend + DB
/backend/           @dhamo-dev
/db/migrations/     @dba-team

# CI and release
/.github/workflows/ @devops-team
```

7.10 Branch Protection Rules

Protect the main branch from accidental pushes and force pushes. GitHub → Settings → Branches → Add branch protection rule. In a company this is **mandatory**.

Rule	Why
Require a pull request before merging	No direct push to main
Require approvals (1–2)	Four-eyes principle
Dismiss stale approvals	New commits need a fresh review
Require status checks	Merge only when CI is green
Require conversation resolution	All comments resolved
Require linear history	No merge commits (optional)
Block force pushes / deletions	History stays safe
Include administrators	Leads follow the rules too

MERGE QUEUE

If PR traffic is high, enable the **merge queue**. GitHub queues the PR, reruns checks against the latest main, and merges only when they pass — avoiding a broken main.

7.11 Issues — Track Work

Bugs, feature requests, and tasks are all issues. Every PR should link an issue: `Closes #128`. When the PR merges, the issue closes automatically. Use the issue number in the branch name: `fix/128-empty-search`.

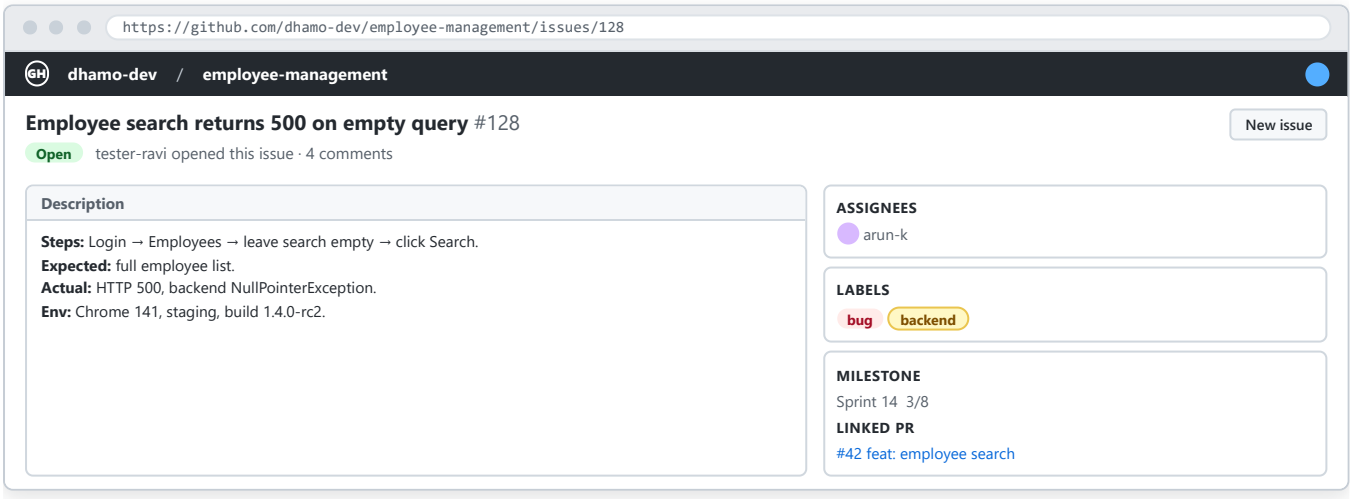


Fig 7.2 ILLUSTRATIVE MOCKUP **Issue detail page.** Steps, expected/actual, labels, assignee, milestone, and linked PR — no back-and-forth needed.

7.12 Labels & Milestones

Labels are categories (bug, feature, frontend, priority). A milestone is a deadline or release bucket (Sprint 14, v1.5). Together they make the issue list easy to filter.

Label set

`bug` · `feature` · `docs` · `frontend` · `backend` · `priority:high` · `good first issue`.

Milestone rules

One issue = one milestone. Set a due date. At sprint end, move open issues to the next sprint so carry-over stays visible.

`.github/ISSUE_TEMPLATE/bug.yml`

```yaml
name: Bug report
description: Report a reproducible defect
title: "[Bug]: "
labels: ["bug", "triage"]
body:
  - type: textarea
    attributes:
      label: Steps to reproduce
      placeholder: 1. Login  2. Open Employees  3. Search empty
  - type: textarea
    attributes:
      label: Expected vs actual
  - type: input
    attributes:
      label: Environment (browser, build, env)
```

DISCUSSION → ISSUE

An open-ended idea starts as a discussion; once it becomes actionable, click "Create issue" to convert it. A discussion is a conversation, while an issue is tracked work with an owner and deadline.

7.13 Wiki & Discussions

Feature	Purpose	Example
Wiki	Long-form docs, versioned pages	Setup guide, architecture notes, runbook
Discussions	Open-ended Q&A, ideas, announcements	"Which state manager should we use?"
README	Quick start, first impression	Install, run, test, contribute
Issues	Actionable, closable work items	Bug #128, feature #131

REAL-TIME IT COMPANY USAGE

Typical flow: the tester opens a bug issue → the tech lead adds a priority label and sprint milestone → the developer creates a `fix/128-...` branch and opens a PR → reviewer approves and CI turns green → squash merge → the issue closes automatically. Every week the team reviews stale PRs and issues.

Practice: fork a repo, add upstream, make a change on a feature branch, and open a PR. Try the PR template, a linked issue, a draft PR, and a reviewer request.

CHAPTER 08 · CI/CD AUTOMATION

GitHub Actions & CI/CD

When you push, build, test, scan, and deploy happen automatically — hand the manual work to the machine.

12 topics | Level: **Intermediate** | Time: **110 min** | Most asked: **DevOps rounds**

In a company, "the code merged" is not enough — the tests must pass, the build must succeed, the security scan must be clean, and the app must deploy. All of this has to happen automatically. That is **CI/CD**; in GitHub, the tool for it is **GitHub Actions**.

8.1 CI/CD in One Minute

CI

Continuous Integration: build + tests + lint on every push. Catches bugs early.

CD (Delivery)

Keeps the artifact ready; deploy happens after manual approval. Common in enterprises.

CD (Deployment)

Deploys to production automatically as soon as checks pass. Common in startups and web apps.

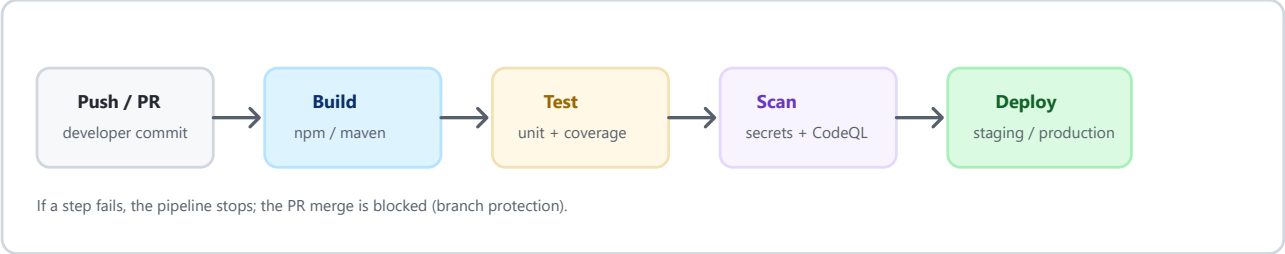


Fig 8.1 **Typical CI/CD pipeline.** A push triggers build → test → scan → deploy. Any red step stops the line.

8.2 Actions Building Blocks

Term	Meaning
Workflow	YAML file in `.github/workflows/`
Event	Trigger: push, pull_request, schedule, manual
Job	One unit of work; own runner; parallel by default
Step	Command or action inside a job; sequential
Action	Reusable package (checkout, setup-node, upload-artifact)
Runner	VM that executes the job (ubuntu, windows, mac)

8.3 Your First Workflow

Create `.github/workflows/ci.yml` at the repo root. When you push, a run starts in the Actions tab.

.github/workflows/ci.yml

```yaml
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci && npm test
```

READ IT LIKE A SENTENCE

"**On** a push/PR to main, **run** the build **on** ubuntu: checkout, Node 20, npm ci, test." YAML uses spaces, not tabs — 2 spaces per level.

8.4 Triggers — When Should It Run?

.github/workflows/ci.yml

```yaml
on:
  push:
    branches: [main, release/*]
  pull_request:
    branches: [main]
  workflow_dispatch:        # manual Run workflow button
  schedule:
    - cron: "0 2 * * *"     # daily 02:00 UTC
```

COMMON MISTAKE

With only `on: push`, PR checks are not enforced — you need the `pull_request` event. Path filter: `paths: ["frontend/**"]`.

8.5 Jobs & Steps

.github/workflows/ci.yml

```yaml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run lint
  test:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test
```

needs

`needs: lint` → the test job waits for lint.

Parallel vs serial

Jobs run in parallel; steps are serial.

8.6 Runners — Where It Runs

Runner	Spec (public repos)	Use for
`ubuntu-latest`	4 vCPU, 16 GB RAM, 14 GB SSD	Default; Docker, Node, Java
`windows-latest`	4 vCPU, 16 GB RAM	.NET, Windows installers
`macos-latest`	3 vCPU, 7 GB RAM	iOS, macOS builds
Self-hosted	Your own machine/VPC	Private network, GPU, compliance

MINUTES & COST

Private repos have a free-minutes limit; macOS minutes cost 10×. Use caching (`cache: npm`). Never run untrusted PR code on a self-hosted runner — it is a security risk.

8.7 Marketplace Actions

For common work, use a ready-made action instead of writing your own script. Pin the version (`@v4`); do not use `@main` (supply-chain risk).

.github/workflows/ci.yml

```yaml
- uses: actions/checkout@v4
- uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: 21
- uses: docker/build-push-action@v6
```

PIN BY SHA

In high-security repos, pin with `uses: owner/action@<full-commit-sha>`. A tag is mutable; a SHA is immutable. Dependabot updates GitHub Actions versions automatically.

8.8 Node CI — Complete Example

.github/workflows/node-ci.yml

```yaml
name: Node CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/
```

8.9 Matrix Builds

Run the same job across multiple versions and OS combinations in parallel. A must for library maintainers: Node 18/20/22 on Ubuntu/Windows/Mac.

.github/workflows/matrix.yml

```yaml
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
        node: [18, 20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm ci && npm test
```

COST WARNING

2 OS × 3 versions = 6 jobs. With `fail-fast: false`, all combinations still run when one fails (easier debugging, higher cost). Use `include` / `exclude` to control the combinations.

8.10 Artifacts — Keep Build Output

The runner is deleted when the job ends. Upload build files, test reports, and screenshots as **artifacts**; the default retention is 90 days. Use artifacts to share files between jobs.

.github/workflows/build.yml

```yaml
- uses: actions/upload-artifact@v4
  with:
    name: app-build
    path: dist/
    retention-days: 14
- uses: actions/download-artifact@v4
  with:
    name: app-build
    path: dist/
```

8.11 Secrets & Variables

Never write API keys, database passwords, or deploy tokens in code. Add them in Repo → Settings → Secrets and variables → Actions. Use `${{ secrets.NAME }}` in the workflow; GitHub masks them in logs automatically.

Secret

Encrypted and masked. `secrets.DEPLOY_TOKEN`. Fork PRs do not receive secrets (safe).

Variable

Plain configuration: `vars.API_URL`. Use for environment-specific values.

8.12 Deploy Workflow

.github/workflows/deploy.yml

```yaml
name: Deploy
on:
  push:
    tags: ["v*"]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: actions/deploy-pages@v4
```

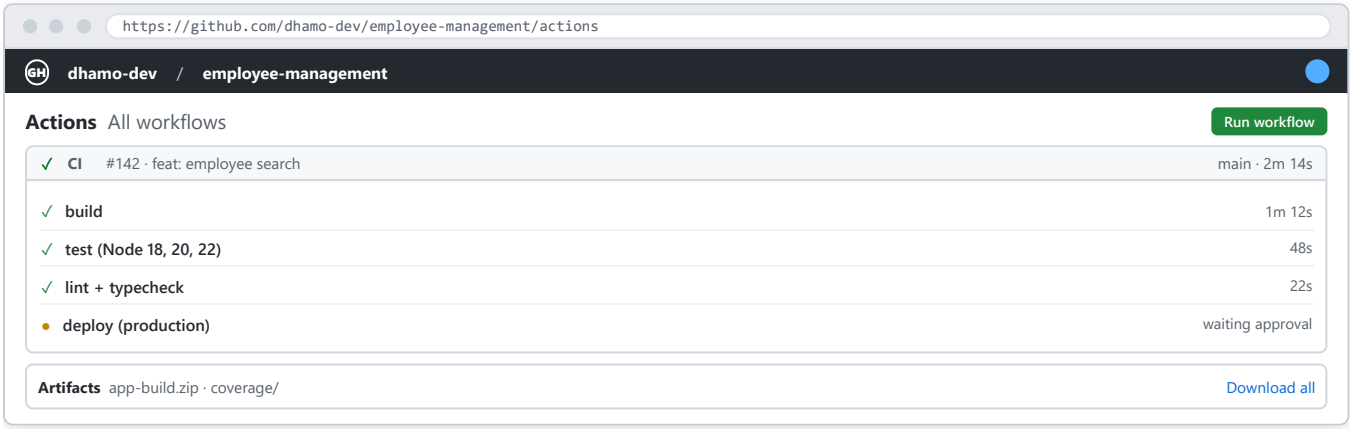


Fig 8.2 ILLUSTRATIVE MOCKUP **Actions run summary.** Jobs, durations, status, and artifacts. The production deploy waits for approval.

REAL-TIME IT COMPANY USAGE

In a product team, CI runs automatically when a PR opens; with "status checks required" branch protection, a red CI blocks the merge. A merge to main deploys to staging, and a tag push deploys to production. Secrets are kept at the environment level — developers never see the production token.

Practice: create `ci.yml` in a simple Node repo. Push and watch the run in the Actions tab. Break one test, observe the red run, then fix it and make it green.

CHAPTER 09 · ACCOUNT & SUPPLY CHAIN

GitHub Security

Protect your account, manage tokens, scan dependencies, and prevent secret leaks — production-grade habits.

10 topics | Level: **Essential** | Time: **90 min** | Audit: **Mandatory**

One leaked token in a company repo can mean a full breach. In this chapter we look practically at locking down an account, connecting a machine, scanning dependencies, and blocking secrets. Security is the responsibility of the feature team, not just "DevOps team work".

9.1 Two-Factor Authentication (2FA)

A password alone is not enough. 2FA means a password plus a second proof (authenticator app, passkey, hardware key). In GitHub: **Settings → Password and authentication**; org owners can enforce it. Print your recovery codes and keep them safe.

TOTP app
Microsoft/Google Authenticator. A 6-digit code that rotates every 30s. Easiest option.

Passkey
Fingerprint/Face ID. Phishing-resistant, no code to type.

Security key
YubiKey USB/NFC. Highest assurance for admins.

RECOVERY CODES
Lose your phone and the account locks. Download the recovery codes and keep them in a password manager. SMS 2FA is weak — SIM swap attacks exist; prefer TOTP or a passkey.

9.2 SSH Keys — Passwordless Identity

An SSH key pair is a private key (on your laptop, never shared) plus a public key (on GitHub). It is better than an HTTPS token for daily work. One key pair cannot be used for multiple GitHub accounts — you need a key per account.

terminal

```
ssh-keygen -t ed25519 -C "dhamo@example.com"
# Press Enter 3x: default path, empty passphrase (or a strong one)
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub      # copy to GitHub → SSH keys
ssh -T git@github.com
```

MULTIPLE ACCOUNTS
Create a Host alias in `~/.ssh/config`: `Host github-work` → `HostName github.com`, `IdentityFile ~/.ssh/id_work`. Remote: `git@github-work:org/repo.git`.

9.3 Personal Access Tokens (PAT)

For HTTPS pushes, API calls, and CI scripts, use a PAT instead of a password. GitHub → Settings → Developer settings → Personal access tokens. A **fine-grained** token (repo-specific, chosen read/write permissions) is safer than a classic one.

Type	Scope	Use
Fine-grained	One owner, selected repos, selected permissions	Recommended for scripts/CI
Classic	All repos, broad scopes (`repo`, `admin`)	Legacy only
GitHub App	Installation token, short-lived	Org-wide automation
OIDC	No stored secret; cloud trusts GitHub	CI → AWS/Azure deploy

NEVER COMMIT A TOKEN

If a token is committed, GitHub push protection blocks the push; but a token that was already pushed is **compromised**. Immediately revoke → rotate → clean history (BFG/filter-repo). Set an expiry: 30–90 days.

9.4 Secrets Management

There are three levels: Actions secret, environment secret, and org secret. Keep the production deploy token as an **environment** secret with required reviewers; developers never see the value.

Do

Fine-grained token · short expiry · environment protection · OIDC for cloud · rotate on offboarding · audit access.

Don't

Token in code · committing `.env` · secrets in PR logs · personal token in shared CI · the same token for all environments.

`.github/workflows/deploy.yml`

```yaml
- run: ./deploy.sh
  env:
    API_TOKEN: ${{ secrets.API_TOKEN }}
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123:role/gh-deploy
    aws-region: ap-south-1
```

9.5 Dependabot — Dependency Updates

If an old npm/Maven package has a CVE, Dependabot opens an alert and an automated PR. Use a weekly schedule, grouped updates, and auto-merge for patch versions.

`.github/dependabot.yml`

```yaml
version: 2
updates:
  - package-ecosystem: npm
    directory: /frontend
    schedule: { interval: weekly }
    open-pull-requests-limit: 5
    groups:
      react: { patterns: ["react*"] }
  - package-ecosystem: maven
    directory: /backend
    schedule: { interval: weekly }
```

9.6 CodeQL & Code Scanning

CodeQL is semantic code analysis. It finds patterns such as SQL injection, XSS, path traversal, and hardcoded credentials. **Security tab → Code scanning → Default setup** is the easiest; an advanced workflow allows custom queries.

.github/workflows/codeql.yml

```yaml
name: CodeQL
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
  schedule: [{ cron: "0 3 * * 1" }]
jobs:
  analyze:
    runs-on: ubuntu-latest
    permissions: { security-events: write, contents: read }
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with: { languages: javascript, java }
      - uses: github/codeql-action/autobuild@v3
      - uses: github/codeql-action/analyze@v3
```

ALERT TRIAGE

When you **dismiss** an alert, give a reason (false positive / won't fix / used in tests). A new alert on a PR fails the check — blocking the merge. Keep open alerts at zero in the weekly security review.

9.7 Secret Scanning & Push Protection

GitHub scans more than 100 service patterns (AWS, Stripe, Slack). **Push protection** blocks a secret before it is pushed. You can add custom patterns for team-internal tokens.

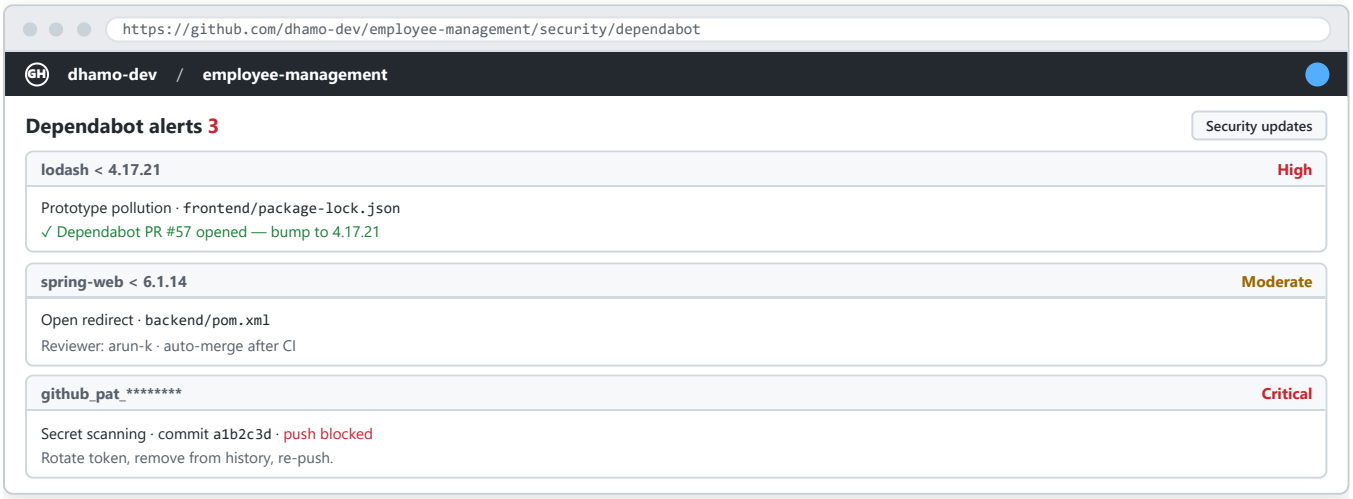


Fig 9.1 ILLUSTRATIVE MOCKUP **Security alerts.** Dependabot CVEs + secret scanning block — all in one Security tab.

9.8 Security Advisories

If your repo has a vulnerability, it can be reported **privately**: Security → Advisories → New draft advisory. The maintainer fixes it and publishes a patched version; you can request a CVE. This is responsible disclosure in open source.

1. **Private report**
   Keep the discussion private with the reporter; do not create a public issue (exploit details would leak).
2. **Assess & fix**
   List affected versions, the patched version, severity (CVSS), and CWE. Keep the fix branch private.
3. **Publish advisory**
   After the patched release is published, make the advisory public and request a CVE ID.
4. **Notify users**
   Release notes + advisory link + upgrade command. The Dependabot alert updates automatically.

9.9 Signed Commits & Vigilant Mode

Cryptographic proof that you really made the commit. Sign with a GPG or SSH key; GitHub shows a "Verified" badge. In vigilant mode, unsigned commits are explicitly shown as "Unverified".

terminal

```
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global gpg.format ssh
git commit -S -m "feat: signed commit"
git log --show-signature -1
```

BRANCH PROTECTION

With "Require signed commits" enabled, an unsigned commit cannot merge. It is mandatory in regulated teams (finance/health). SSH signing has been supported by GitHub since 2022.

9.10 Least Privilege & Audit Log

Give people only the access they need. Repo roles: **Read < Triage < Write < Maintain < Admin**. Contractors get read; a CI bot gets write on a specific repo. Org → Settings → Audit log records every sensitive action.

Control	Action
Team-based access	Use team permissions instead of adding individuals
Audit log	Export to SIEM; alert on repo visibility change
IP allow list	Office/VPN-only access for enterprise
Offboarding	Remove org member, revoke PAT/SSH, rotate deploy keys

REAL-TIME IT COMPANY USAGE

In a company, 2FA is enforced org-wide; PAT expiry is 90 days; Dependabot, CodeQL, and secret scanning are all on by default. Production deploys go through OIDC; there are no long-lived AWS keys. Quarterly access review plus audit log export.

Practice: enable 2FA, add an SSH key and test with `ssh -T`, create a fine-grained PAT and set an expiry. In a sample repo, add a Dependabot config and watch the alert/PR flow.

CHAPTER 10 · PLANNING & TRACKING

GitHub Projects

Drag issues onto a board, plan a sprint, and see team progress on one screen.

9 topics | Level: **Team lead** | Time: **70 min** | Tool: **Projects v2**

A GitHub-native workflow that works as an alternative to Jira/Linear. Issues + Projects + Milestones together are enough for a small or medium team. In this chapter we create a board and cover custom fields, automation, and sprint reporting.

10.1 Projects — What & Why

A project is a flexible view layer over issues and PRs. The same data can be seen as a board, a table, or a roadmap. The issue lives in the repo; in the project it becomes a **card**. You can combine issues from multiple repos in one project.

Board
Kanban columns by status. Best for the daily standup.

Table
Spreadsheet view: priority, size, assignee, sprint.

Roadmap
Timeline by start/target date. Leadership review.

10.2 Create & Link a Project

1. **Create**
   Profile/repo/org → Projects → New project → choose the Board/Table/Roadmap template.
2. **Link repo**
   Project → ⋮ → Settings → Manage access → link repositories. Issues are added automatically.
3. **Add items**
   Add existing issues, or click `+` to create a new issue. A PR also becomes a card.
4. **Set status**
   Todo / In Progress / In Review / Done. Dragging a card updates the status field automatically.

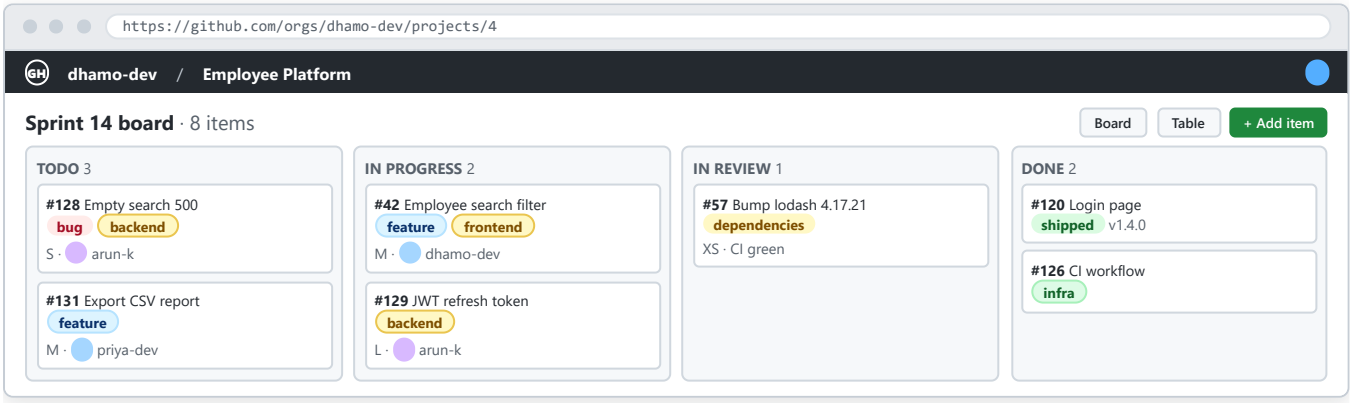


Fig 10.1 ILLUSTRATIVE MOCKUP **Sprint board.** Status columns, issue cards, labels, size, assignee — a single source of truth for standup.

10.3 Board View — Daily Flow

A board is a set of columns grouped by a field (usually Status). A WIP limit keeps the team focused: if In Progress is above 3, do not start new work. Clicking a card opens the issue detail in a side panel.

STANDUP SCRIPT
Read from left to right: "Yesterday #42 moved to review, today I start #131, blocker is #128 waiting on a backend fix." A standup without a board update is wasted.

10.4 Table & Roadmap Views

The same project data through different lenses. In the table you can sort, filter, and group; in the roadmap you can see start/target dates on a timeline. Name each view and share it with the team.

View	Best for	Key control
Board	Daily execution, WIP	Group by Status
Table	Backlog grooming, estimates	Sort by Priority, Size
Roadmap	Quarter planning, stakeholders	Start date + Target date

10.5 Custom Fields

Status, Assignee, and Labels are default. What teams need: **Priority** (P0–P3), **Size** (XS–XL), **Sprint** (iteration), **Estimate** (number), and **Target date**. Once added, a field can be used in the table, board, and roadmap.

Useful fields
Priority · Size · Sprint · Estimate · Target date · Team · Environment.

Anti-pattern
10+ fields, and nobody fills them. Start with 4; review monthly. To enforce required fields, add a validation step to the workflow.

10.6 Built-in Automation

Project → ⋮ → Workflows. Enable common automation without writing code. Item added, closed, PR merged, review requested — each can trigger a status or field update.

Workflow	Effect
Item added to project	Status = Todo
Issue closed	Status = Done
PR merged	Linked issue Status = Done
Auto-add to project	Repo issue/PR automatically add
Item reopened	Status = Todo

terminal

```
gh project list --owner dhamo-dev
gh project item-add 4 --owner dhamo-dev --url https://github.com/dhamo-dev/app/issues/131
gh project field-list 4 --owner dhamo-dev
```

10.7 Issue Templates & Forms

Standard bug/feature templates speed up triage. Create YAML forms in `.github/ISSUE_TEMPLATE/`; you can enforce required fields. Disable blank issues in the config.

.github/ISSUE_TEMPLATE/config.yml

```yaml
blank_issues_enabled: false
contact_links:
  - name: Security report
    url: https://github.com/dhamo-dev/app/security/advisories/new
    about: Do not open a public issue for vulnerabilities
```

TEMPLATE = TRIAGE SPEED

The bug form requires steps, expected, actual, and environment. The feature form requires problem, proposed solution, and acceptance criteria. Without a template, you get issues that just say "not working" — a waste of time.

10.8 Task Lists & Sub-Issues

Break a big issue into small tasks. A task list is a markdown checklist; sub-issues track parent-child progress automatically. Link a PR with "Closes #128" and the task completes.

Issue body — task list

```markdown
## Acceptance criteria
- [x] API accepts empty query
- [x] Returns full list, not 500
- [ ] Frontend empty state
- [ ] Tests cover empty + special chars
- [ ] Deployed to staging
```

SUB-ISSUES

In the parent issue detail, click "Create sub-issue". Each sub-issue has its own assignee/status; the parent progress bar updates automatically. Ideal for the epic pattern.

10.9 Insights & Sprint Reporting

Project → Insights: burn-up, status distribution, velocity. For the weekly review, three numbers matter: **throughput** (closed items), **WIP**, and **carry-over**. If you use story points, look at the velocity chart.

Burn-up

Completed vs total scope. Scope creep becomes visible.

Throughput

Weekly closed issues. The trend matters, not the absolute number.

Cycle time

Days from start to done. Use it to find the bottleneck.

REAL-TIME IT COMPANY USAGE

A team uses repo issues + an org project board + a sprint milestone. Daily standup happens on the board; weekly grooming happens in the table; monthly leadership review happens on the roadmap. Automation updates status — no manual status email.

Practice: create a project, link a repo, and add 5 issues. Create board/table/roadmap views, add Priority + Size fields, and enable the "Issue closed → Done" workflow.

CHAPTER 11 · FREE HOSTING

GitHub Pages

Portfolio, docs, landing page — publish a free static website straight from a repo.

7 topics | Level: **Beginner+** | Time: **60 min** | Cost: **Free**

Need a live portfolio link for recruiters? Want to host project docs professionally? GitHub Pages serves static files for free from a CDN. It is free for public repos; private repos need a paid plan (check your org policy).

11.1 What Are GitHub Pages?

Static site hosting: HTML, CSS, JS, images. Server-side code (Node/Java/PHP) does not run — for that you need Render/Railway/Fly.io. You can use a build step (React/Vite/Jekyll); only the static output files are served.

Project site
`user.github.io/repo` — a subpath for the repo.

User site
`user.github.io` — repo named `user.github.io`.

Custom domain
`www.example.com` — CNAME + DNS.

11.2 Deploy from a Branch

1. **Push static files**
   Put `index.html` in the repo root or in a `/docs` folder.
2. **Open settings**
   Repo → Settings → Pages → Build and deployment.
3. **Select source**
   Deploy from a branch → branch `main` → folder `/ (root)` or `/docs` → Save.
4. **Wait & visit**
   The URL is live in 1–2 minutes. Watch the "pages build and deployment" run in the Actions tab.

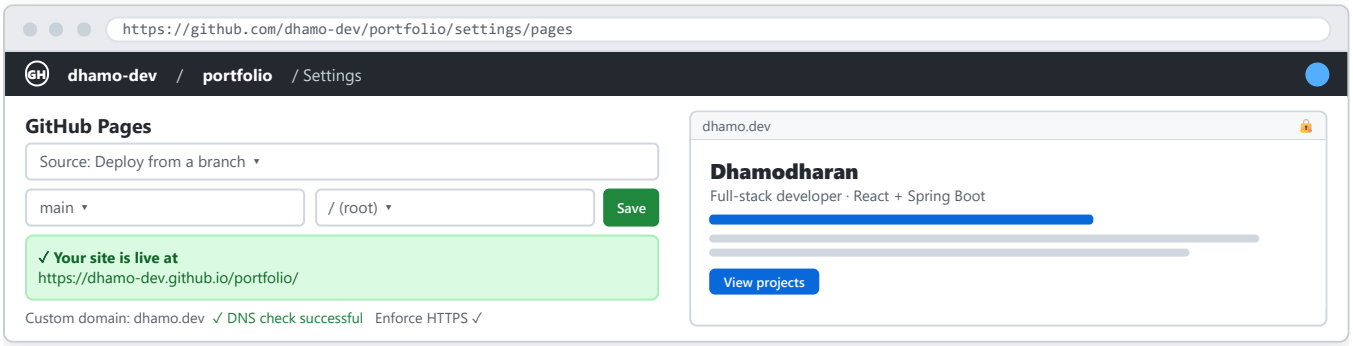


Fig 11.1 ILLUSTRATIVE MOCKUP **Pages settings + live site.** Branch source, custom domain, HTTPS, and the deployed portfolio.

11.3 Deploy with GitHub Actions

If you need a build step (Vite, Next export, Jekyll), use an Actions workflow. Settings → Pages → Source = **GitHub Actions**. The workflow builds and uploads the artifact.

.github/workflows/pages.yml

```yaml
name: Deploy Pages
on:
  push: { branches: [main] }
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci && npm run build
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with: { path: dist }
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: github-pages
    steps:
      - uses: actions/deploy-pages@v4
```

SPA ROUTING

A React Router BrowserRouter deep link (e.g. `/about`) returns 404 on refresh. Put a copy of index.html at `dist/404.html`, or use HashRouter.

11.4 Custom Domain

1. **DNS records**
   Apex domain `example.com`: A records to GitHub Pages IPs. `www`: CNAME to `user.github.io`.
2. **Add in GitHub**
   Settings → Pages → Custom domain → enter domain → Save. Wait until the DNS check passes.
3. **CNAME file**
   For branch deploys, commit a `CNAME` file (domain name only) at the repo root — the custom domain is retained.
4. **Verify**
   Check with `dig example.com` / `nslookup`. Propagation can take up to 24h.

11.5 HTTPS & Security

When you add a custom domain, GitHub issues a Let's Encrypt certificate automatically (a few minutes). Enable the **Enforce HTTPS** checkbox in Settings — HTTP requests are redirected to HTTPS. Watch out for mixed content (http:// images/scripts).

11.6 Jekyll Static Site

Jekyll is a Ruby static site generator. Markdown + Liquid templates become HTML. GitHub Pages builds Jekyll by default (branch source); no custom workflow is needed. Ideal for blogs and docs.

_config.yml

```
title: Dhamo Dev Notes
description: React, Spring Boot, Git
theme: minima
baseurl: "/notes"
url: "https://dhamo-dev.github.io"
markdown: kramdown
plugins:
  - jekyll-feed
  - jekyll-seo-tag
```

NO RUBY LOCALLY?

You can edit and commit on GitHub — Pages will build it. For a local preview, use `bundle exec jekyll serve` (requires Ruby + Bundler). To override theme files, use a `_layouts/` folder.

11.7 React SPA on Pages

You can build a Vite React app as static files and host it. Set `base` in `vite.config.js` (project site subpath), then push/upload the build output from `dist/`.

vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  base: "/employee-management/",   // repo name
});
```

ENVIRONMENT NOTE

Pages is static only. API calls need an external backend (Render/Railway). Allow CORS; put the API URL in a build-time environment variable (`VITE_API_URL`). Never put secrets in the frontend bundle — everything is public.

REAL-TIME IT COMPANY USAGE

Teams use Pages for internal docs: architecture decision records, API docs, and onboarding guides. It is free hosting for portfolio/demo sites. Pages is not used for a production customer app — that needs auth, SSR, and an API, so it goes on a cloud platform.

Practice: push a simple index.html to a repo and enable Pages. Set a custom domain (or username.github.io) and enforce HTTPS. Deploy a Vite React build to Pages from `dist/`.

CHAPTER 12 · CAREER & CRAFT

Professional GitHub

Profile, commits, docs, releases, CLI — the signals that show an interviewer you are a serious developer.

12 topics | Level: **Career** | Time: **80 min** | Impact: **High**

Code that works is only half the job; you also have to look professional. Clear commit messages, semantic versions, a changelog, docs, a clean repo — these habits build trust in a team. A recruiter judges your GitHub profile in 30 seconds.

12.1 Profile README

If the repo name matches your username (`dhamo-dev/dhamo-dev`), it renders as your profile page. Role, stack, current projects, contact — a one-screen summary. Pin up to 6 repos.

dhamo-dev/README.md

```
### Hi, I'm Dhamodharan

Full-stack developer — React, Spring Boot, PostgreSQL.

- Currently building: employee-management (React + Spring Boot)
- Learning: GitHub Actions, Kubernetes
- Portfolio: https://dhamo.dev
- Contact: dhamo@example.com

**Stack:** Java 21 · Spring Boot · React · TypeScript · Docker
```

RECRUITER CHECKLIST

Profile photo · one-line bio · 3–6 pinned real projects · each repo README with a screenshot + run steps · live demo link. Don't pin tutorial clones.

12.2 Contribution Graph

Daily commits show up as green squares. It is a consistency signal, but **don't chase meaningless commits** — recruiters don't look at commit count; they look at quality, tests, and docs. You can enable private contribution counts.



Fig 12.1 ILLUSTRATIVE MOCKUP **Profile contribution graph.** Daily consistency; empty weeks are normal — vacation, focus work.

12.3 Open Source Contribution

1. **Start small**
   Filter by the `good first issue` / `documentation` label.
2. **Read CONTRIBUTING**
   Setup steps, branch naming, test command, DCO/CLA requirements.
3. **Claim & discuss**
   Comment "I'd like to work on this" on the issue; confirm your approach.
4. **PR with tests**
   A small focused PR, tests pass, respond to maintainer review. Be patient — it can take days or weeks.

12.4 Commit Message Convention

Conventional Commits = `type(scope): summary`. Changelog generation, semantic release, readable history — all of it comes from this. Subject 50 chars, imperative mood ("add" not "added"), explain the why in the body.

terminal

```
feat(search): add empty-query handling
fix(auth): refresh token before expiry
docs(readme): add local setup steps
refactor(api): extract employee mapper
test(search): cover special characters
```

Type	Meaning
`feat`	New user-facing feature (MINOR)
`fix`	Bug fix (PATCH)
`docs`	Documentation only
`refactor`	No behavior change
`test`	Tests added/fixed

BREAKING CHANGE

`feat(api)!: remove legacy endpoint` or footer `BREAKING CHANGE: ...` → MAJOR version bump. Add a commitlint hook + PR title check in CI to enforce the convention.

12.5 Semantic Versioning

`MAJOR.MINOR.PATCH` — the library/API versioning standard. Breaking change → MAJOR; backward-compatible feature → MINOR; bug fix → PATCH. `0.y.z` = initial development, API unstable.

Change	Version	Example
Breaking API change	2.3.1 → **3.0.0**	Remove a field, change the response shape
New feature	2.3.1 → **2.4.0**	Add a filter endpoint
Bug fix	2.3.1 → **2.3.2**	Fix a null pointer
Pre-release	**3.0.0-rc.1**	Release candidate

12.6 Changelog

`CHANGELOG.md` = a human-readable release history. "Keep a Changelog" format: Added, Changed, Deprecated, Removed, Fixed, Security. Give the tag date; latest at the top.

CHANGELOG.md

```
## [1.4.0] - 2026-11-09
### Added
- Employee search filter with empty-query handling (#128)
- CSV export endpoint (#131)

### Fixed
- JWT refresh race condition on tab switch (#126)

### Security
- Bump lodash to 4.17.21 (CVE-2021-23337)
```

12.7 Review Etiquette

Comment style
Critique the code, not the person. "This query runs per row" > "you wrote bad code". Give a suggestion + reason. `nit:` = non-blocking.

Response style
Agree / explain / push back respectfully. "Good catch, fixed in 3f2a1b" or "Keeping it because...". Use the resolve button.

24-HOUR RULE
A PR needs a first review within 24 hours of opening; reviewers get 2 working days. If you're blocked, raise it in standup — a silent PR is a team bottleneck.

12.8 Documentation & README

README = the project's front door. A new developer should be able to run it in 10 minutes. Screenshot, prerequisites, setup commands, env variables, test command, one-line architecture, contribution link.

README.md

```
# Employee Management

React + Spring Boot CRUD app with JWT login.

## Quick start
1. `docker compose up -d db`
2. `cd backend && ./mvnw spring-boot:run`
3. `cd frontend && npm ci && npm run dev`
4. Open http://localhost:5173

## Environment
Copy `.env.example` to `.env` and fill DB_URL, JWT_SECRET.

## Tests
`./mvnw test` · `npm test`
```

12.9 .gitignore

Generated files, secrets, and IDE files should not be committed. Use a language-specific template (github/gitignore). To ignore a file that is already tracked you need `git rm --cached file`.

.gitignore

```
node_modules/
dist/
target/
.env
.env.*
!.env.example
*.log
.idea/
.vscode/
.DS_Store
coverage/
```

12.10 Repository Structure

Predictable structure = fast onboarding. In a monorepo, separate `frontend/` + `backend/` + `infra/`. At the root: README, LICENSE, .gitignore, CONTRIBUTING, CHANGELOG.

repository tree

```
employee-management/
├── frontend/          React + Vite
├── backend/           Spring Boot
├── infra/             Docker, compose, k8s
├── docs/              ADRs, API docs
├── .github/           workflows, templates, CODEOWNERS
├── README.md
├── CHANGELOG.md
├── CONTRIBUTING.md
└── LICENSE
```

NAMING RULES

lowercase-with-hyphens for files/folders; `PascalCase` components; `camelCase` Java methods? No — Java `camelCase`, classes `PascalCase`, constants `UPPER_SNAKE`. Document the team convention.

12.11 Gists

Share a small snippet/note — no need for a full repo. Secret gist = unlisted (not private!), public gist = searchable. Useful for code snippets, config samples, and interview notes; use a repo for project code.

Use	Tool
One-file snippet	Gist
Full project	Repository
Long docs	Repo + Pages / Wiki
Quick question	Discussion / Stack Overflow

12.12 GitHub CLI (gh)

Manage repos, PRs, issues, Actions, and releases from the terminal. Powerful for scripting; avoids browser context switching. Install: `winget install GitHub.cli`, then `gh auth login`.

terminal

```
gh repo clone dhamo-dev/employee-management
gh pr create --fill
gh pr checks 42
gh pr review 42 --approve
gh issue list --label bug
gh run list --limit 5
gh release create v1.4.0 --generate-notes
```

REAL-TIME IT COMPANY USAGE

Teams use commitlint + semantic-release: version + changelog automatically from the merge commit message. Docs repos maintain ADRs (Architecture Decision Records). CLI scripts are common in CI/CD.

Practice: create a profile README, pin 3 repos. Use a Conventional Commit title in one PR. Add a CHANGELOG entry. Try `gh pr create --fill`.

CHAPTER 13 · REAL PROJECT SIMULATION

Real-Time IT Company Workflow

From sprint start to production release on a React + Spring Boot team — end to end.

12 topics | Team: **6 members** | Sprint: **2 weeks** | Level: **Job-ready**

This is the final integration chapter. You know every command, but in a real team you need to know **who does what, and when**. Let's use an employee management app (React frontend + Spring Boot backend) as the team example.

13.1 Team Roles & Ownership

Role	Member	Owns
Tech Lead	Karthik	Architecture, PR final approval, release, production hotfix
Backend	Arun	Employee API, JWT, DB migrations, backend tests
Backend	Meera	Search/report API, validation, performance
Frontend	Priya	React components, routing, state, accessibility
Frontend	Dhamo	Search UI, API integration, E2E tests
Tester	Ravi	Test cases, bug reports, staging verification, release sign-off

13.2 Sprint Flow — Two Weeks

1. **Planning (Day 1)**
   The TL grooms the backlog; the team estimates; issues move to the sprint milestone; owners are assigned.

2. **Development (Day 2–8)**
   Feature branch → small commits → draft PR early → daily push → CI green.

3. **Review & QA (Day 8–9)**
   Reviewer approval, tester verifies on staging, bug issues created, fixes on the same branch.

4. **Release (Day 10)**
   The TL creates the tag, production deploy, smoke test, release notes, retrospective.

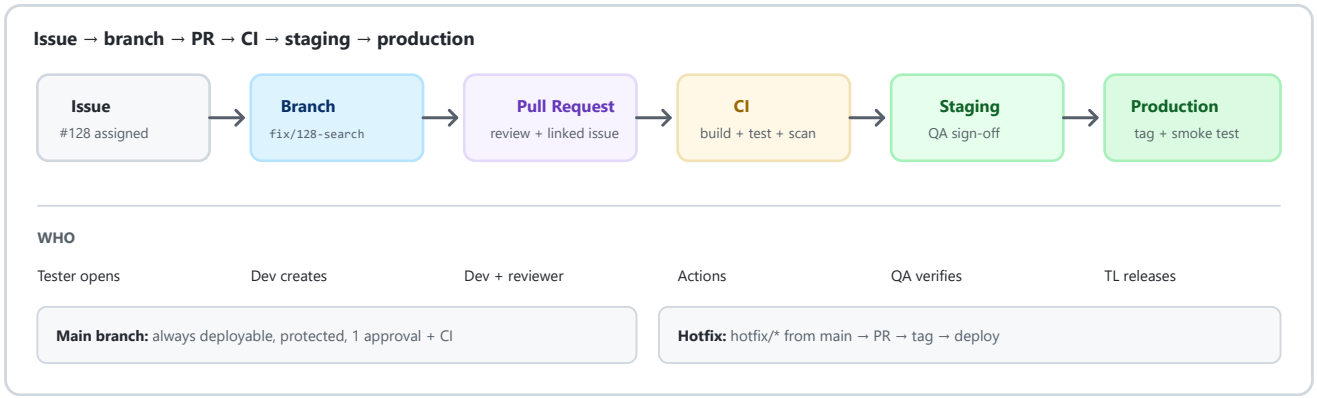


Fig 13.1 **Team delivery workflow.** Every change takes the same path — no shortcuts, no direct main commits.

13.3 Issue Assignment

The tester opens the bug; the TL triages it and sets priority, owner, and sprint. The developer comments "I'll take this" and moves the status to In Progress. This avoids duplicate work.

Issue #128 — assignment

```
Title: Employee search returns 500 on empty query
Assignee: arun-k
Labels: bug, backend, priority:high
Milestone: Sprint 14
Branch: fix/128-empty-search
```

13.4 Branch Naming Standard

Pattern	Example
feature/<issue>-slug	feature/131-csv-export
fix/<issue>-slug	fix/128-empty-search
hotfix/<version>	hotfix/1.4.1-payment
chore/<slug>	chore/update-spring-boot

terminal

```
git switch main
git pull --ff-only
git switch -c fix/128-empty-search
```

13.5 React Frontend Flow

1. **Start from main**
   Create a branch, `npm ci`, `npm run dev`. Create the component + test file.
2. **API contract**
   Check the backend swagger/openapi; start with mock data. Use the `VITE_API_URL` env.
3. **Test & lint**
   `npm test`, `npm run lint`. Cover empty/loading/error states.
4. **Draft PR early**
   Attach a screenshot; give the reviewer UI context. The tester verifies on the preview URL.

13.6 Spring Boot Backend Flow

1. **Branch & DB**
   Create a branch, start local Postgres, add a migration script (Flyway/Liquibase).
2. **Code layers**
   Controller → Service → Repository. Add validation + exception handler.
3. **Tests**
   Unit + integration tests: `./mvnw test`. Cover empty query, SQL injection, and auth cases.
4. **PR + API doc**
   Update Swagger. In the backend PR, include migration order + rollback note.

13.7 PR Flow — Feature to Merge

1. **Push branch**
   `git push -u origin fix/128-empty-search` → click "Compare & pull request" on GitHub.
2. **Fill template**
   What/why/test steps; `Closes #128`; screenshots; set reviewer + labels + milestone.
3. **CI green**
   Wait until build, unit, lint, and CodeQL pass. If it fails, fix on the same branch + push.
4. **Review**
   Address reviewer comments; re-request review. The merge button is enabled only after approval.
5. **Squash & merge**
   Keep the PR title clean and squash merge. Delete branch. Issue auto-closes. Staging deploy triggers.

terminal

```
git add src/EmployeeService.java
git commit -m "fix(search): guard empty query"
git push
gh pr create --fill --reviewer karthik-tl
gh pr checks 42
```

13.8 Review & Fix Cycle

Reviewer finds
Null check missing · test case missing · N+1 query · hardcoded URL · sensitive log line.

Author fixes
New commit on the same branch · reply "fixed in abc123" · resolve conversation · re-request review.

DON'T FORCE-PUSH DURING REVIEW
Reviewer comments lose their commit context. Add new commits; if you squash on merge, the history stays clean. Force push only your own unreviewed branch.

13.9 Conflict Handling in Team

Main moves fast; if your branch is 3 days old you'll hit a conflict. **Pull/rebase daily.** Don't panic when a conflict appears — understand both changes and combine them.

terminal

```
git fetch origin
git rebase origin/main
# resolve conflicts in VS Code
git add <resolved-files>
git rebase --continue
git push --force-with-lease
```

TEAM PROTOCOL
If you're unsure during conflict resolution, ping the original author. Don't blindly select "ours/theirs". Run the tests locally after resolving.

13.10 CI Tests — Merge Gate

A PR runs unit + integration + lint + security scan. Make these checks required in branch protection. Red CI blocks the merge — this avoids a "fix it later" culture.

.github/workflows/pr.yml

```yaml
name: PR Checks
on: { pull_request: { branches: [main] } }
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 21 }
      - run: cd backend && ./mvnw verify
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: cd frontend && npm ci && npm test -- --run
```

13.11 Deploy Pipeline

1. **Merge to main**
   Staging auto-deploys. The tester runs a smoke test + regression.
2. **Tag release**
   The TL pushes the `v1.4.0` tag. The production deploy workflow triggers.
3. **Approval gate**
   GitHub Environment "production" required reviewers; the TL approves.
4. **Smoke test**
   Health endpoint, login, search, one critical flow check. If it fails, roll back.

ROLLBACK READY
Before deploying, have the previous tag + DB migration rollback plan ready. If you have a feature flag, you can turn the deploy off and stay in control.

13.12 Release & Hotfix

terminal

```bash
# scheduled release
git switch main && git pull
git tag -a v1.4.0 -m "Sprint 14 release"
git push origin v1.4.0

# production hotfix
git switch -c hotfix/1.4.1-payment main
# fix + test + PR + merge
git tag -a v1.4.1 -m "Fix payment timeout"
git push origin v1.4.1
```

13.13 Sprint Simulation — Day by Day

Day	What happens	Git/GitHub action
1	Planning, estimate, assign	Issues → Sprint 14 milestone, owners
2	Arun starts the backend branch	`fix/128-empty-search`, draft PR
3	Priya starts the search UI branch	`feature/131-csv-export`, mock API
5	Mid-sprint sync	Board update, blockers, CI status
7	Backend PR review	2 comments, fixes, approval
8	Squash merge + staging	Merge, branch delete, auto deploy
9	QA regression + sign-off	Bug issues, fixes, re-test
10	Release + retro	Tag v1.4.0, release notes, deploy

13.14 Definition of Done

Code
Reviewed + approved · tests added · lint clean · no secrets · migration tested · docs updated.

Delivery
CI green · staging verified · release note · monitoring dashboard · rollback plan.

TEAM ANTI-PATTERNS
Direct push to main · PR without linked issue · "LGTM" review without reading · red CI merge · force push shared branch · secrets in code · 2-week-old branch · release without tag.

13.15 Metrics That Matter

Lead time
Issue start → production. Target: days, not weeks.

Review time
PR open → first review. Target: < 24h.

Change failure
Deploy fail %. Target: < 15%, rollback fast.

REAL-TIME IT COMPANY USAGE
This flow is the standard in product companies: protected main, short branches, PR template + CODEOWNERS, CI required, staging auto-deploy, production approval + tag, weekly release. A GitHub-native workflow is enough for teams of 5–8; enterprises layer Jira/Azure DevOps on top, but the Git flow is the same.

Final practice: simulate a 6-member team. Create one issue → assign → branch → PR → review → CI → merge → tag → release notes. Run the full cycle end-to-end in one repo.

APPENDIX A · QUICK REFERENCE

Command Cheat Sheet

Daily commands, all on one page. Bookmark it and keep it beside your terminal.

2 pages | Level: **All** | Use: **Daily**

A.1 Setup & Configuration

terminal

```
git config --global user.name "Dhamodharan"
git config --global user.email "dhamo@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase true
git config --list --show-origin
git init
git clone https://github.com/user/repo.git
```

A.2 Daily Workflow

Command	What it does
`git status`	Working tree + staging state
`git diff` / `git diff --staged`	Unstaged / staged changes
`git add <file>` / `git add -p`	Stage file / stage hunks
`git commit -m "feat: ..."`	Commit staged changes
`git commit --amend --no-edit`	Add files to last local commit
`git log --oneline --graph --all`	Visual history
`git blame <file>`	Line-wise author + commit

A.3 Branch & Merge

terminal

```
git branch -a                # list local + remote
git switch -c feature/login  # create + switch
git merge feature/login      # merge into current
git merge --no-ff feature/x  # always merge commit
git merge --abort            # cancel conflicted merge
git branch -d feature/login  # delete merged branch
git push origin --delete feature/login
```

A.4 Remote & Sync

Command	What it does
`git remote -v`	Show remote URLs
`git remote add upstream <url>`	Add original repo (fork)
`git fetch origin`	Download refs, don't merge
`git pull --ff-only`	Fetch + fast-forward only
`git push -u origin main`	Push + set upstream
`git push --force-with-lease`	Safe force push (own branch)

A.5 Undo & Recovery

Command	Use when
`git restore file`	Discard unstaged file changes
`git restore --staged file`	Unstage, keep working change
`git reset --soft HEAD~1`	Undo commit, keep staged
`git reset --mixed HEAD~1`	Undo commit + unstage
`git reset --hard HEAD~1`	Discard commit + changes
`git revert <hash>`	Undo pushed commit safely
`git reflog`	Find lost commits
`git stash push -m "wip"`	Park unfinished work
`git stash pop`	Restore + remove stash

A.6 Advanced

terminal

```
git rebase -i HEAD~3           # squash/reword/drop
git rebase origin/main         # update feature base
git cherry-pick 8f2c1ab        # copy one commit
git tag -a v1.0.0 -m "release" # annotated tag
git push origin v1.0.0
git worktree add ../hotfix hotfix/1.0.1
git bisect start               # binary search bug
git lfs track "*.psd"
```

A.7 GitHub & CLI

Command / action	Purpose
`gh auth login`	Authenticate CLI
`gh repo clone owner/repo`	Clone repo
`gh pr create --fill`	PR from current branch
`gh pr checks`	CI status
`gh pr review --approve`	Approve PR
`gh issue list --label bug`	Filter issues
`gh run list`	Recent workflow runs
`gh release create v1.4.0`	Create release

A.8 Danger Commands

THINK TWICE

`reset --hard` · `push --force` · `clean -fdx` · `branch -D` · `stash clear` · `filter-repo` · `rm -rf .git`. Never force push a shared branch; stash uncommitted work first with `git stash`.

APPENDIX B · TROUBLESHOOTING

Common Errors & Fixes

Read the error message, don't panic. These are the 14 repeat offenders — cause and exact fix.

14 errors | Level: **All** | Use: **When stuck**

Error message	Why it happens	Exact fix
`fatal: not a git repository`	Wrong folder, or repo not initialised	`cd` into project, then `git init`
`Permission denied (publickey)`	SSH key missing or not added to GitHub	`ssh -T git@github.com`, add `~/.ssh/id_ed25519.pub`
`Authentication failed`	Old password cached; GitHub needs a token	Clear Windows Credential Manager, use a PAT
`src refspec main does not match any`	No commit yet, or branch named `master`	`git add -A` + `git commit`, then `git branch -M main`
`Updates were rejected … fetch first`	Remote has commits your clone lacks	`git pull --rebase`, resolve, then `git push`
`CONFLICT (content): Merge conflict`	Same lines changed on both branches	Edit markers, `git add <file>`, `git commit`
`You have not concluded your merge`	A merge stopped midway (`MERGE_HEAD`)	Finish it, or `git merge --abort`
`Your local changes would be overwritten`	Dirty working tree blocks switch/pull	Commit, or `git stash` → switch → `git stash pop`
`You are in 'detached HEAD' state`	Checked out a commit or tag, not a branch	`git switch -c my-branch` to keep work
`fatal: bad object / unknown revision`	Typo, or commit never fetched	`git fetch --all`, verify with `git log --all --oneline`
`error: remote origin already exists`	Remote added twice	`git remote set-url origin <url>`
`LF will be replaced by CRLF`	Windows line endings (warning, not fatal)	`git config core.autocrlf true` + `.gitattributes`
`GH013: protected branch`	Direct push blocked by ruleset	Push a branch and open a PR instead
`refusing to merge unrelated histories`	Two repos with separate first commits	`git pull origin main --allow-unrelated-histories`

1 First-aid commands

`git status` → what state am I in?
`git log --oneline -5` → what did I just do?
`git remote -v` → am I pushing to the right place?
`git reflog` → where did that commit go?

2 Read the message properly

fatal = Git stopped. **error** = action failed. **warning** = it continued. **hint** = Git is telling you the next command — read the last three lines first.

NEVER DO THIS WHILE DEBUGGING

When an error appears, don't reach for `git reset --hard` or `rm -rf .git`. First run `git stash` or create a backup branch (`git branch backup-today`) — then experiment freely. Never force push a shared branch.

APPENDIX C · INTERVIEW PREPARATION

Git & GitHub Interview Q&A

12 questions that actually get asked — short answers first, then the detail the interviewer wants.

12 questions | Level: **Junior → Mid** | Format: **Q + model answer**

Q1 What is the difference between Git and GitHub?

Git is a distributed version control system installed on your machine. GitHub is a cloud platform that hosts Git repositories and adds pull requests, issues, Actions and permissions. Git works offline; GitHub needs a network.

Q2 Merge vs rebase — when do you use each?

`merge` keeps history and adds a merge commit; use it for shared branches and PR integration. `rebase` replays commits on a new base for a linear history; use it on your own unpushed feature branch. Never rebase a shared branch.

Q3 Explain `reset --soft`, `--mixed` and `--hard`.

All move the branch pointer. `--soft` keeps changes staged, `--mixed` (default) keeps them unstaged, `--hard` discards working-tree changes. Use `--hard` only on commits you are sure you do not need.

Q4 Reset vs revert — which is safe on a pushed branch?

`revert` is safe: it creates a new commit that undoes an old one, so history is not rewritten. `reset` rewrites history and then needs a force push, which breaks teammates' clones.

Q5 What is the difference between `fetch` and `pull`?

`fetch` downloads remote refs but does not touch your working tree. `pull` is `fetch` plus `merge` (or `rebase` if configured). Fetch first when you want to inspect changes safely.

Q6 How do you resolve a merge conflict?

Run `git status` to list conflicted files, edit the `<<<<<<<` / `=======` / `>>>>>>>` blocks, remove the markers, test, then `git add` the files and `git commit`. Use `git merge --abort` to bail out.

Q7 What happens when you open a pull request?

GitHub compares your branch with the base branch, shows the diff, and starts checks (CI, review apps). Reviewers comment and approve, required checks must pass, then the PR is merged by merge commit, squash or rebase.

Q8 What are branch protection rules?

Rules that stop direct pushes to important branches. Typical settings: require a PR, 1–2 approvals, passing status checks, up-to-date branch, conversation resolution, and no force pushes or deletions.

Q9 What is `cherry-pick` and when is it useful?

It copies one specific commit onto the current branch. Useful for a hotfix that must go to both `main` and a release branch without merging the whole feature.

Q10 How does `git reflog` help recover lost work?

Reflog records every position of HEAD, including commits removed by reset or rebase. Find the hash, then `git branch rescue <hash>` or `git reset --hard <hash>`.

Q11 What is CI/CD, and where does GitHub Actions fit?

CI builds and tests every change automatically; CD deploys the tested artifact. Actions is GitHub's workflow engine — YAML in `.github/workflows` triggered by push, PR or schedule.

Q12 Lightweight vs annotated tags? Fork vs clone?

Annotated tags store a message, tagger and date — use them for releases; lightweight tags are just pointers. A clone copies a repo to your machine; a fork is your own server-side copy used to propose changes to someone else's repo.

ANSWER LIKE A DEVELOPER

After you finish the definition, give **one real example**: "In my last project we rebased feature branches daily and protected `main` with one approval plus CI." That sentence proves you have used it, not just memorised it.

APPENDIX D · HANDS-ON PRACTICE

Practice Tasks & 7-Day Plan

Half the book is done. Now practice at the keyboard — 10 graded tasks, 7 days.

10 tasks | Plan: **7 days** | Level: **Beginner → Advanced**

D.1 Ten Graded Tasks

#	Task	Level	You prove
1	Init a repo, make 3 commits, push to GitHub	Beginner	Local → remote
2	Stage a file, unstage it, discard a change	Beginner	Staging control
3	Create a deliberate conflict on two branches and resolve it	Intermediate	Merge confidence
4	Delete a branch, then recover it with `reflog`	Intermediate	Safe recovery
5	Rebase a feature branch and squash 3 commits into 1	Intermediate	Clean history
6	Open a PR with a template, linked issue and screenshots	Intermediate	Team communication
7	Add a GitHub Actions workflow that runs tests on every PR	Advanced	CI automation
8	Protect `main`: 1 approval, green CI, no force push	Advanced	Repo governance
9	Tag `v1.0.0` and publish release notes	Intermediate	Release flow
10	Cherry-pick a hotfix from `main` to a release branch	Advanced	Production reflexes

D.2 Seven-Day Study Plan

1. **Day 1 — Foundations**
   Ch 1–2. Install, configure, init, stage, commit. Task 1.
2. **Day 2 — History & branches**
   Ch 3–5. Log, diff, branch, merge. Tasks 2 & 3.
3. **Day 3 — Undo & recover**
   Ch 6. Stash, reset, revert, reflog, tags. Tasks 4 & 9.
4. **Day 4 — GitHub collaboration**
   Ch 7. Remote, fork, PR review, issues. Task 6.
5. **Day 5 — Automation**
   Ch 8–9. Actions, checks, Dependabot, secrets. Task 7.
6. **Day 6 — Team simulation**
   Ch 13. Two-person PR flow, protection, release. Tasks 5, 8 & 10.
7. **Day 7 — Polish & interview**
   Ch 10–12, appendices. Profile README, mock Q&A, portfolio repo.

PRACTICE RULES

Always use a **scratch repo** for destructive experiments. Break it on purpose, then fix it. Reading commands gives knowledge; typing them gives skill. Daily 45 minutes > one 6-hour weekend session.

CONCLUSION

You Are Now Git-Ready

13 chapters, one real workflow, and a habit that will follow you through every job.

You have now walked the full path from `git init` to a production release. Not just the commands — the **why** and **when** too. That is the main difference that makes someone a developer.

1 Think in commits
One logical change, one commit. Small commits make review easy, revert safe and history readable.

2 Branch small, merge often
Short-lived branches + early PRs = fewer conflicts. Long branches are where merges go to die.

3 Automate the boring part
CI, branch protection and templates protect the team from mistakes — including your own.

Your 30-Day Plan

1. **Week 1 — Fluency**
   Daily commit in a practice repo. Tasks 1–4. No copy-paste — type every command.
2. **Week 2 — History skills**
   Rebase, squash, cherry-pick, reflog. Break a branch and recover it twice.
3. **Week 3 — Collaboration**
   One PR per day with a real description. Review someone else's code politely and specifically.
4. **Week 4 — Portfolio**
   Publish one project with README, CI badge, release tag and a clean commit history. That repo is your proof.

Keep Learning

Resource	Why it helps
Pro Git (free book)	The complete reference for internals and edge cases
GitHub Docs	Current UI, Actions syntax, security and API behaviour
GitHub Skills	Interactive courses inside a real repository
Oh My Git!	A game that makes branch and merge mental models visual
GitHub CLI manual	`gh` commands for PRs, issues and releases without leaving the terminal

ONE LAST THING

Tools change, but the habit stays: **pull before you start, commit small, push daily, review kindly, and never hide a mistake.** Git records everything — that is exactly why it is safe to experiment. Break things in a branch, learn, and ship.

Happy committing!

Dhamodharan · Developer Learning Series